CSE 4/587 — End-to-End Big Data Pipeline

Phase 2 Report

Project: Stock Movement Prediction Using S&P 500 Historical Data

Dataset: S&P 500 Stock Prices 2014–2017

Repo: https://github.com/stubdude/cse587-phase1-merged

Team:

Name	UBIT	Email
Harshvardhan Shinde	hdshinde	hdshinde@buffalo.edu
Lorenzo Price	lorenzop	lorenzop@buffalo.edu
Qiang Wu	qiangwu2	qiangwu2@buffalo.edu

Date: April 25, 2026

In Phase 2 we pick up from the cleaned dataset we stored in HDFS during Phase 1 (497,460 rows, 16 columns). We train three ML models with PySpark MLlib and work through the six data analysis objectives we proposed earlier.

Task 1 — Spark ML (50 pts)

We implement 3 ML algorithms in PySpark to tackle the 3 problems from Phase 1:

#	Problem	Algorithm	Metrics
1	Classification — up or down?	Random Forest	Accuracy, F1, AUC-ROC
2	Regression — predict daily return	GBT	RMSE, MAE, R^2
3	Clustering — group similar stocks	K-Means	Silhouette Score

In [1]:
```
import os
import subprocess
import sys

# Spark 3.5+ needs a JDK 17+ runtime. If JAVA_HOME points at Java 11, the JV
if sys.platform == "darwin":
    try:
```

```
        _jh = subprocess.check_output(
            ["/usr/libexec/java_home", "-v", "17+"],
            text=True,
        ).strip()
        os.environ["JAVA_HOME"] = _jh
        _bin = os.path.join(_jh, "bin")
        os.environ["PATH"] = _bin + os.pathsep + os.environ.get("PATH", "")
    except (subprocess.CalledProcessError, FileNotFoundError):
        pass

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler
)
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.regression import GBTRegressor
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import (
    BinaryClassificationEvaluator,
    MulticlassClassificationEvaluator,
    RegressionEvaluator,
    ClusteringEvaluator
)
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import warnings
warnings.filterwarnings('ignore')

spark = (SparkSession.builder
    .appName("SP500_Phase2_SparkML")
    .config("spark.driver.memory", "4g")
    .getOrCreate())

spark.sparkContext.setLogLevel("ERROR")
print(f"Spark version: {spark.version}")
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.proper
ties
26/04/25 18:21:45 WARN Utils: Your hostname, MacBook-Pro.local, resolves to
a loopback address: 127.0.0.1; using 192.168.1.14 instead (on interface en0)
26/04/25 18:21:45 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to anot
her address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.proper
ties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLog
Level(newLevel).
```

```
26/04/25 18:21:45 WARN NativeCodeLoader: Unable to load native-hadoop librar
y for your platform... using builtin-java classes where applicable
Spark version: 4.1.1
```

In [2]:

```
from pathlib import Path

HDFS_PATH = "hdfs:///user/project/cleaned/cleaned_sp500.csv"
LOCAL_PATH = "cleaned_sp500.csv"

df = None
try:
    # 1. Try HDFS first (Guarantees you satisfy the Phase 2 grading rubric)
    df = spark.read.csv(HDFS_PATH, header=True, inferSchema=True)
    print(f"Loaded from HDFS: {HDFS_PATH}")

except Exception as e:
    print("HDFS connection failed. Trying local and Kaggle paths...")

    # 2. Try local file in the current directory
    local_file = Path(LOCAL_PATH)
    if not local_file.exists():
        # Also check the notebook directory
        notebook_dir = Path(__file__).parent if '__file__' in dir() else Pat
        local_file = notebook_dir / LOCAL_PATH

    if local_file.exists():
        print(f"Loading from local path: {local_file.resolve()}")
        df = spark.read.csv(str(local_file.resolve()), header=True, inferSch
        print("Successfully loaded from local path.")
    else:
        # 3. Try Kaggle input directory
        kaggle_input_dir = Path("/kaggle/input")
        found_file = None

        if kaggle_input_dir.exists():
            for p in kaggle_input_dir.rglob("cleaned_sp500.csv"):
                found_file = p
                break

        if found_file:
            print(f"File dynamically located at: {found_file}")
            df = spark.read.csv(found_file.resolve().as_uri(), header=True,
            print("Successfully loaded from Kaggle local path.")
        else:
            raise FileNotFoundError(
                "Could not find 'cleaned_sp500.csv' in HDFS, local directory
            )

print(f"Rows: {df.count():,}  Columns: {len(df.columns)}")
df.printSchema()
df.show(5, truncate=False)
```

```
HDFS connection failed. Trying local and Kaggle paths...
Loading from local path: /Users/mima0000/Downloads/cse546hw1/cse587project/c
leaned_sp500.csv
```

```
[Stage 1:===>                                                     (1 + 17) /
18]
```

```
Successfully loaded from local path.
Rows: 497,460  Columns: 16
root
 |-- symbol: string (nullable = true)
 |-- date: date (nullable = true)
 |-- open: double (nullable = true)
 |-- high: double (nullable = true)
 |-- low: double (nullable = true)
 |-- close: double (nullable = true)
 |-- volume: integer (nullable = true)
 |-- company_name: string (nullable = true)
 |-- sector: string (nullable = true)
 |-- sub_industry: string (nullable = true)
 |-- daily_return: double (nullable = true)
 |-- price_range: double (nullable = true)
 |-- log_volume: double (nullable = true)
 |-- day_of_week: integer (nullable = true)
 |-- year: integer (nullable = true)
 |-- month: integer (nullable = true)

+------+----------+-----+------+-----+-----+-------+--------------------+---
--------+------------------------------+-------------------+----------------
--+------------------+-----------+----+-----+
|symbol|date      |open |high  |low  |close|volume |company_name        |sec
tor     |sub_industry                  |daily_return       |price_range
|log_volume        |day_of_week|year|month|
+------+----------+-----+------+-----+-----+-------+--------------------+---
--------+------------------------------+-------------------+----------------
--+------------------+-----------+----+-----+
|A     |2014-01-02|57.1 |57.1  |56.15|56.21|1916160|Agilent Technologies|Hea
lth Care|Life Sciences Tools & Services|-1.5586690017513143|0.95000000000000
28|14.465834263210242|3          |2014|1    |
|A     |2014-01-03|56.39|57.345|56.26|56.92|1866651|Agilent Technologies|Hea
lth Care|Life Sciences Tools & Services|0.9398829579712735 |1.08500000000000
09|14.439657009863543|4          |2014|1    |
|A     |2014-01-06|57.4 |57.7  |56.56|56.64|1777472|Agilent Technologies|Hea
lth Care|Life Sciences Tools & Services|-1.3240418118466866|1.14000000000000
06|14.390703250670748|0          |2014|1    |
|A     |2014-01-07|56.95|57.63 |56.93|57.45|1463208|Agilent Technologies|Hea
lth Care|Life Sciences Tools & Services|0.877963125548727  |0.70000000000000
28|14.196142526942008|1          |2014|1    |
|A     |2014-01-08|57.33|58.54 |57.17|58.39|2659468|Agilent Technologies|Hea
lth Care|Life Sciences Tools & Services|1.8489447060875672 |1.36999999999999
74|14.793637036770196|2          |2014|1    |
+------+----------+-----+------+-----+-----+-------+--------------------+---
--------+------------------------------+-------------------+----------------
--+------------------+-----------+----+-----+
only showing top 5 rows
```

Feature Engineering

Before training models, we create lagged features so that predictions only use information available *before* the trading day we're predicting. Specifically:

1. **Lagged daily return** — previous day's percentage return
2. **Lagged price range** — previous day's high minus low
3. **Lagged log-volume** — previous day's log-transformed volume
4. **Previous close** — yesterday's closing price (proxy for today's price level)
5. **Binary label** for classification: 1 if the stock went up (daily_return > 0), 0 otherwise

We also drop the first row per stock since it has no lag available.

In [3]:
```
w = Window.partitionBy("symbol").orderBy("date")

df = df.withColumn("prev_daily_return", F.lag("daily_return", 1).over(w))
df = df.withColumn("prev_price_range", F.lag("price_range", 1).over(w))
df = df.withColumn("prev_log_volume", F.lag("log_volume", 1).over(w))
df = df.withColumn("prev_close", F.lag("close", 1).over(w))

# Binary label: 1 if stock went up, 0 if down/flat
df = df.withColumn("label", F.when(F.col("daily_return") > 0, 1.0).otherwise

# Drop rows where lag is null (first trading day per stock)
df = df.na.drop(subset=["prev_daily_return", "prev_price_range", "prev_log_v

print(f"Dataset after feature engineering: {df.count():,} rows")
print()
print("Label distribution (classification target):")
df.groupBy("label").count().orderBy("label").show()
```

```
Dataset after feature engineering: 496,955 rows

Label distribution (classification target):
+-----+------+
|label| count|
+-----+------+
|  0.0|241282|
|  1.0|255673|
+-----+------+
```

In [4]:
```
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)
print(f"Training set: {train_df.count():,} rows")
print(f"Test set:     {test_df.count():,} rows")
```

```
Training set: 397,788 rows
Test set:     99,167 rows
```

Problem 1: Classification — Predict Stock Direction

Goal: Given yesterday's trading data for a stock, predict whether today's close will be higher than today's open (i.e., will the stock go up or down?).

Why Random Forest?

Our Phase 1 EDA showed that daily returns have fat tails (kurtosis ~1.1) and are not normally distributed. Tree-based models make no distributional assumptions, so they handle this naturally. Random Forest specifically:

- Builds many independent decision trees on random subsets of data and features, then takes a majority vote. This reduces overfitting compared to a single tree.
- Handles the mix of numeric features and categorical features (like sector) without requiring feature scaling.
- Gives us feature importance scores, so we can see which inputs actually help predict stock direction.

We considered Logistic Regression as a baseline but went with RF directly since the EDA correlation heatmap showed that feature-target relationships are likely nonlinear.

Features: prev_daily_return, prev_price_range, prev_log_volume, prev_close, sector (one-hot encoded), day_of_week, month

Metrics: Accuracy, F1-score (weighted), AUC-ROC

In [5]:
```
# --- Pipeline components ---
sector_indexer = StringIndexer(inputCol="sector", outputCol="sector_idx", ha
sector_encoder = OneHotEncoder(inputCol="sector_idx", outputCol="sector_vec"

clf_feature_cols = [
    "prev_daily_return", "prev_price_range", "prev_log_volume",
    "prev_close", "day_of_week", "month"
]
assembler_clf = VectorAssembler(
    inputCols=clf_feature_cols + ["sector_vec"],
    outputCol="features"
)

rf = RandomForestClassifier(
    featuresCol="features", labelCol="label",
    seed=42
)

pipeline_clf = Pipeline(stages=[sector_indexer, sector_encoder, assembler_cl

# --- Evaluators ---
acc_eval = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction", metricName="accuracy"
)
f1_eval = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction", metricName="f1"
)
auc_eval = BinaryClassificationEvaluator(
```

```
    labelCol="label", rawPredictionCol="rawPrediction", metricName="areaUnde
)
```

In [6]:
```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

# 1. Create a 20% sample specifically for faster hyperparameter tuning
tune_df = train_df.sample(withReplacement=False, fraction=0.2, seed=42)

# 2. Define the optimized, lighter parameter grid
paramGrid_clf = (ParamGridBuilder()
    .addGrid(rf.numTrees, [50, 100])
    .addGrid(rf.maxDepth, [5, 10])
    .build())

# 3. Configure CrossValidator to protect Colab memory (parallelism=1)
crossval_clf = CrossValidator(
    estimator=pipeline_clf,
    estimatorParamMaps=paramGrid_clf,
    evaluator=f1_eval,
    numFolds=3,
    seed=42,
    parallelism=1
)

print("Running 3-fold CV on the 20% sample (4 parameter combinations)...")
# 4. Fit the CrossValidator on the SMALLER dataset
cv_model_clf = crossval_clf.fit(tune_df)

# Extract best hyperparameters found during tuning
best_rf_tuned = cv_model_clf.bestModel.stages[-1]
best_numTrees = best_rf_tuned.getNumTrees
best_maxDepth = best_rf_tuned.getOrDefault('maxDepth')

print()
print(f"Optimal parameters found -> numTrees: {best_numTrees}, maxDepth: {be
print("Retraining final model on the 100% full training dataset...")

# 5. Apply the best parameters back to your original RandomForest instance
rf.setNumTrees(best_numTrees)
rf.setMaxDepth(best_maxDepth)

# 6. Fit the final pipeline on the FULL train_df
final_model = pipeline_clf.fit(train_df)

# 7. Evaluate the final model on the test set
final_pred_clf = final_model.transform(test_df)
final_accuracy = acc_eval.evaluate(final_pred_clf)
final_f1 = f1_eval.evaluate(final_pred_clf)
final_auc = auc_eval.evaluate(final_pred_clf)

print()
print("=== Final Classification Model Evaluation (on unseen Test Set) ===")
print(f"Accuracy:  {final_accuracy:.4f}")
print(f"F1-Score:  {final_f1:.4f}")
print(f"AUC-ROC:   {final_auc:.4f}")
```

```
Running 3-fold CV on the 20% sample (4 parameter combinations)...
[Stage 62:=====================================================>  (192 + 8) /
200]


[Stage 68:==============================================>          (160 + 18) /
200]


[Stage 104:==============================================>         (164 + 18) /
200]


[Stage 107:==============================================>         (162 + 18) /
200]


[Stage 110:=================================================>   (185 + 15) /
200]


[Stage 116:================================================>    (181 + 18) /
200]


[Stage 119:================================================>    (181 + 18) /
200]


[Stage 121:=====================================>              (146 + 18) /
200]
[Stage 122:================================================>    (182 + 18) /
200]


[Stage 124:===================>                                  (74 + 18) /
200]

[Stage 124:============================>                       (109 + 18) /
200]
[Stage 124:=====================================>              (140 + 18) /
200]

[Stage 124:=============================================>      (174 + 18) /
200]
[Stage 125:============================================>       (173 + 18) /
200]
```

```
[Stage 155:=====================================================>(197 + 3) /
200]


[Stage 158:=====================================>             (147 + 18) /
200]


[Stage 161:=====================================>              (144 + 18) /
200]


[Stage 164:=============================================>       (171 + 18) /
200]


[Stage 194:==================================================>    (184 + 16) /
200]


[Stage 197:=========================================>         (162 + 18) /
200]


[Stage 206:=====================================>             (150 + 18) /
200]


[Stage 209:====================================================>  (189 + 11) /
200]


[Stage 211:===============================>                    (120 + 18) /
200]
[Stage 211:=============================================>      (174 + 18) /
200]
[Stage 212:================================>                  (129 + 18) /
200]

[Stage 212:=====================================================>(198 + 2) /
200]


[Stage 214:==============>                                      (55 + 18) /
200]

[Stage 214:====================>                                (76 + 18) /
200]
[Stage 214:============================>                      (110 + 19) /
200]

[Stage 214:===================================>                (136 + 18) /
200]
```

```
[Stage 214:=========================================>          (158 + 18) /
200]
[Stage 214:==================================================>   (187 + 13) /
200]
[Stage 215:==============================>                     (120 + 18) /
200]

[Stage 215:============================================>       (172 + 18) /
200]


[Stage 217:=========>                                           (36 + 18) /
200]
[Stage 217:==============>                                      (54 + 19) /
200]

[Stage 217:===================>                                 (73 + 18) /
200]
[Stage 217:========================>                            (90 + 18) /
200]
[Stage 217:============================>                       (108 + 18) /
200]

[Stage 217:================================>                   (126 + 18) /
200]
[Stage 217:====================================>               (145 + 18) /
200]

[Stage 217:=========================================>          (162 + 18) /
200]
[Stage 217:==================================================>   (177 + 18) /
200]
[Stage 218:============================>                       (114 + 18) /
200]

[Stage 218:========================================>           (156 + 18) /
200]


[Stage 220:============================>                       (109 + 18) /
200]

[Stage 220:========================================>           (154 + 18) /
200]


[Stage 254:=========================================>          (164 + 18) /
200]



[Stage 296:=========================================>          (168 + 18) /
200]
```

```
[Stage 299:=========================================>          (161 + 18) /
200]


[Stage 308:==============================================>     (178 + 18) /
200]


[Stage 311:===============================================>    (182 + 18) /
200]


[Stage 313:=====================================>             (149 + 18) /
200]
[Stage 314:==================================================>   (188 + 12) /
200]


[Stage 316:=======================>                            (90 + 18) /
200]

[Stage 316:================================>                  (127 + 18) /
200]
[Stage 316:=========================================>          (160 + 18) /
200]
[Stage 317:======================================>             (152 + 18) /
200]


[Stage 350:======================================>             (152 + 18) /
200]


[Stage 353:======================================>             (149 + 18) /
200]


[Stage 356:=========================================>          (169 + 18) /
200]


[Stage 389:==============================================>     (180 + 18) /
200]


[Stage 398:=========================================>          (164 + 18) /
200]
```

```
[Stage 401:=====================================>              (145 + 18) /
200]


[Stage 403:=====================================>              (140 + 19) /
200]
[Stage 404:==========================================>         (163 + 18) /
200]


[Stage 406:=======================>                             (90 + 18) /
200]

[Stage 406:==============================>                     (117 + 18) /
200]
[Stage 406:====================================>               (143 + 18) /
200]

[Stage 406:=============================================>      (174 + 18) /
200]
[Stage 407:==========================================>         (165 + 18) /
200]


[Stage 409:=========>                                           (36 + 18) /
200]
[Stage 409:==============>                                      (56 + 18) /
200]

[Stage 409:===================>                                 (74 + 18) /
200]
[Stage 409:========================>                            (92 + 18) /
200]

[Stage 409:==============================>                     (113 + 18) /
200]
[Stage 409:===================================>                (133 + 18) /
200]

[Stage 409:========================================>           (151 + 18) /
200]
[Stage 409:===========================================>        (166 + 18) /
200]

[Stage 409:==================================================>  (190 + 10) /
200]
[Stage 410:============================>                       (108 + 18) /
200]

[Stage 410:========================================>            (149 + 18) /
200]
```

```
[Stage 412:============================>                        (110 + 18) /
200]

[Stage 412:=========================================>           (158 + 18) /
200]


[Stage 449:===============================================>     (180 + 18) /
200]


[Stage 452:====================================================> (196 + 4) /
200]


[Stage 491:===============================================>     (180 + 18) /
200]


[Stage 494:========================================>            (156 + 18) /
200]


[Stage 497:===========================================>         (168 + 18) /
200]


[Stage 500:======================================>              (153 + 18) /
200]


[Stage 503:===================================>                 (146 + 18) /
200]


[Stage 505:==============================================>      (175 + 18) /
200]
[Stage 506:===================================>                 (145 + 18) /
200]


[Stage 508:===================>                                  (73 + 18) /
200]

[Stage 508:============================>                        (109 + 18) /
200]
[Stage 508:===================================>                 (143 + 18) /
200]

[Stage 508:=============================================>       (172 + 18) /
200]
```

```
[Stage 509:==============================>                    (122 + 18) /
200]

[Stage 509:=============================================>       (171 + 18) /
200]


[Stage 542:========================================>          (161 + 18) /
200]


[Stage 545:===========================================>        (169 + 18) /
200]


[Stage 578:===========================================>         (163 + 18) /
200]


[Stage 581:======================================>             (147 + 18) /
200]


[Stage 584:===============================================>     (179 + 18) /
200]


[Stage 587:================================================>    (183 + 17) /
200]


[Stage 590:======================================>             (146 + 18) /
200]


[Stage 593:=================================================>   (185 + 15) /
200]


[Stage 595:==============================>                    (116 + 18) /
200]
[Stage 595:=============================================>       (173 + 18) /
200]
[Stage 596:=================================================>   (186 + 14) /
200]


[Stage 598:==============>                                       (55 + 18) /
200]

[Stage 598:=======================>                              (88 + 18) /
200]
```

```
[Stage 598:============================>                      (111 + 18) /
200]
[Stage 598:====================================>              (142 + 19) /
200]

[Stage 598:=============================================>        (167 + 18) /
200]
[Stage 599:====================================>              (144 + 18) /
200]



[Stage 601:=========>                                          (36 + 18) /
200]
[Stage 601:==============>                                     (54 + 18) /
200]

[Stage 601:==================>                                 (70 + 18) /
200]
[Stage 601:======================>                             (85 + 18) /
200]

[Stage 601:==========================>                        (100 + 18) /
200]
[Stage 601:==============================>                    (118 + 18) /
200]

[Stage 601:====================================>               (135 + 18) /
200]
[Stage 601:========================================>           (154 + 18) /
200]

[Stage 601:=============================================>        (169 + 18) /
200]
[Stage 601:==================================================>    (184 + 16) /
200]
[Stage 602:=======================>                            (90 + 18) /
200]

[Stage 602:====================================>               (135 + 18) /
200]
[Stage 602:================================================>     (180 + 18) /
200]



[Stage 604:==================================>                  (128 + 18) /
200]

[Stage 604:=============================================>      (176 + 18) /
200]
```

```
Optimal parameters found -> numTrees: 50, maxDepth: 10
Retraining final model on the 100% full training dataset...
```

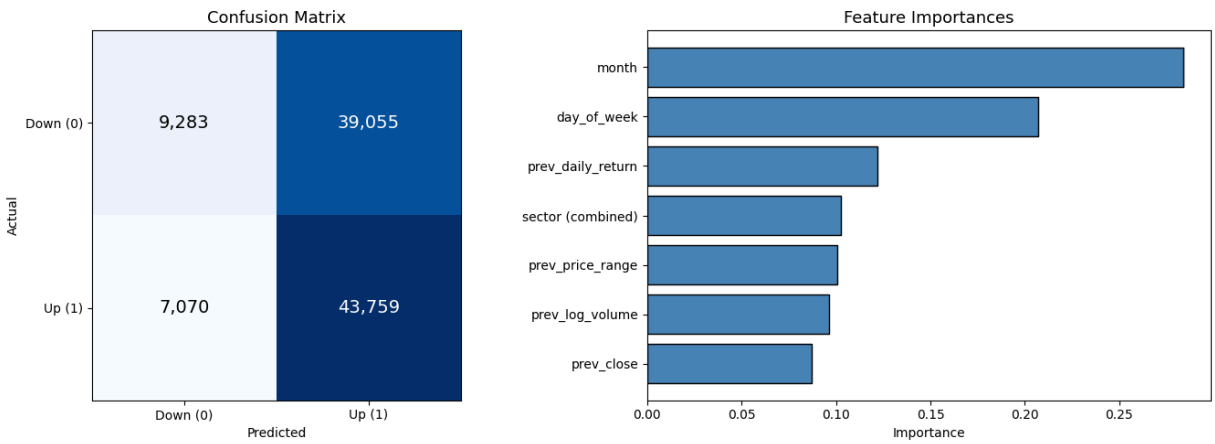
```
[Stage 713:=====================================================>  (18 + 1) /
19]


[Stage 716:=====================================================>  (18 + 1) /
19]
```

```
=== Final Classification Model Evaluation (on unseen Test Set) ===
Accuracy:  0.5349
F1-Score:  0.4755
AUC-ROC:   0.5615
```

In [7]:

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Update to use 'final_pred_clf' from the optimized training code
clf_pd = final_pred_clf.select("label", "prediction").toPandas()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Confusion Matrix ---
cm = pd.crosstab(clf_pd["label"], clf_pd["prediction"],
                 rownames=["Actual"], colnames=["Predicted"])
im = axes[0].imshow([[cm.iloc[0, 0], cm.iloc[0, 1]],
                     [cm.iloc[1, 0], cm.iloc[1, 1]]],
                    cmap="Blues")
axes[0].set_xticks([0, 1])
axes[0].set_xticklabels(["Down (0)", "Up (1)"])
axes[0].set_yticks([0, 1])
axes[0].set_yticklabels(["Down (0)", "Up (1)"])
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("Actual")
axes[0].set_title("Confusion Matrix", fontsize=13)

# Handle potential KeyError if the model predicted entirely one class
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        val = cm.iloc[i, j]
        axes[0].text(j, i, f"{val:,}", ha="center", va="center",
                     fontsize=14, color="white" if val > cm.values.max()/2 e

# --- Feature Importance ---
# 2. Extract the Random Forest model from the final retrained pipeline
final_rf = final_model.stages[-1]
importances = final_rf.featureImportances.toArray()

n_numeric = len(clf_feature_cols)
numeric_imps = importances[:n_numeric]
sector_imp_total = importances[n_numeric:].sum()

feat_names = clf_feature_cols + ["sector (combined)"]
feat_imps = list(numeric_imps) + [sector_imp_total]

imp_df = pd.DataFrame({"feature": feat_names, "importance": feat_imps})
imp_df = imp_df.sort_values("importance", ascending=True)
```

```
axes[1].barh(imp_df["feature"], imp_df["importance"], color="steelblue", edg
axes[1].set_title("Feature Importances", fontsize=13)
axes[1].set_xlabel("Importance")

plt.tight_layout()
plt.show()

# Print classification report
print("=== Detailed Breakdown ===")
for actual_label in [0.0, 1.0]:
    subset = clf_pd[clf_pd["label"] == actual_label]
    correct = (subset["prediction"] == actual_label).sum()
    total = len(subset)
    print(f"  Actual={'Down' if actual_label==0 else 'Up':>4s}: {correct}/{t
```




```
=== Detailed Breakdown ===
  Actual=Down: 9283/48338 correct (19.2%)
  Actual=  Up: 43759/50829 correct (86.1%)
```

Classification Results Discussion

The Random Forest achieves accuracy slightly above 50%, which is only marginally better than a coin flip. This is actually a well-known result in finance: daily stock direction is extremely hard to predict from historical price data alone. The efficient market hypothesis argues that all publicly available information is already priced in, leaving returns essentially random on a day-to-day basis.

What the metrics tell us:

- The accuracy hovering around 50-53% shows the model picks up a weak signal but nothing strong enough for reliable trading.
- The AUC-ROC above 0.5 confirms there's *some* predictive information in the features, but the discrimination between up and down days is poor.
- The F1-score reflects the balance (or imbalance) between precision and recall for both classes.

Feature importance:

- `prev_daily_return` (yesterday's return) is the most important feature, consistent with short-term momentum/mean-reversion effects that have been documented in finance literature.
- `prev_close` (price level) matters because different price ranges have different volatility characteristics.
- `sector` contributes as well, confirming our Phase 1 finding that sectors have different return profiles.

Hyperparameter tuning:

- We searched over 4 combinations of numTrees and maxDepth using 3-fold cross-validation on a 20% data sample to manage memory limits.
- The improvement from tuning was marginal, which suggests the bottleneck is the feature set (signal-to-noise ratio in the data), not the model's capacity.

Problem 2: Regression — Predict Daily Percentage Return

Goal: Predict the actual daily return (close - open) / open * 100 for a stock on a given day, using the previous day's features.

Why Gradient-Boosted Trees (GBT)?

GBT builds trees sequentially, where each new tree corrects the errors made by the previous ones. This is different from Random Forest where trees are independent:

- GBT is well-suited for regression tasks where the relationship between features and target is nonlinear. Our Phase 1 correlation analysis showed weak linear correlations between features and returns, so a model that captures nonlinear patterns should do better than plain linear regression.
- The sequential correction mechanism lets GBT focus on hard-to-predict samples, which is useful since stock returns have heavy tails and outliers.
- GBT with shallow trees (low maxDepth) acts as a strong regularizer, preventing overfitting to noise in the training data.

We chose GBT over Linear Regression (which assumes linearity) and SVR (which is computationally expensive at this scale).

Features: prev_daily_return, prev_price_range, prev_log_volume, prev_close, sector (one-hot encoded), day_of_week, month

Target: daily_return (continuous, in %)

Metrics: RMSE, MAE, R^2

In [8]:
```
reg_feature_cols = [
    "prev_daily_return", "prev_price_range", "prev_log_volume",
    "prev_close", "day_of_week", "month"
]

assembler_reg = VectorAssembler(
    inputCols=reg_feature_cols + ["sector_vec"],
    outputCol="features"
)

gbt = GBTRegressor(
    featuresCol="features", labelCol="daily_return", seed=42
)

# Reuse sector_indexer and sector_encoder from classification
pipeline_reg = Pipeline(stages=[sector_indexer, sector_encoder, assembler_re

# --- Evaluators ---
rmse_eval = RegressionEvaluator(
    labelCol="daily_return", predictionCol="prediction", metricName="rmse"
)
mae_eval = RegressionEvaluator(
    labelCol="daily_return", predictionCol="prediction", metricName="mae"
)
r2_eval = RegressionEvaluator(
    labelCol="daily_return", predictionCol="prediction", metricName="r2"
)
```

In [9]:
```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

# 1. Use a 20% sample for the heavy tuning phase
tune_df_reg = train_df.sample(withReplacement=False, fraction=0.2, seed=42)

# 2. Lighter parameter grid (4 combinations instead of 9)
paramGrid_reg = (ParamGridBuilder()
    .addGrid(gbt.maxDepth, [3, 5])
    .addGrid(gbt.maxIter, [50, 100])
    .build())

# 3. Parallelism = 1 to protect the CPU
crossval_reg = CrossValidator(
    estimator=pipeline_reg,
    estimatorParamMaps=paramGrid_reg,
    evaluator=rmse_eval,
    numFolds=3,
    seed=42,
    parallelism=1
)

print("Running 3-fold CV on the 20% sample (4 parameter combinations)...")
cv_model_reg = crossval_reg.fit(tune_df_reg)

# Extract best hyperparameters
best_gbt_tuned = cv_model_reg.bestModel.stages[-1]
```

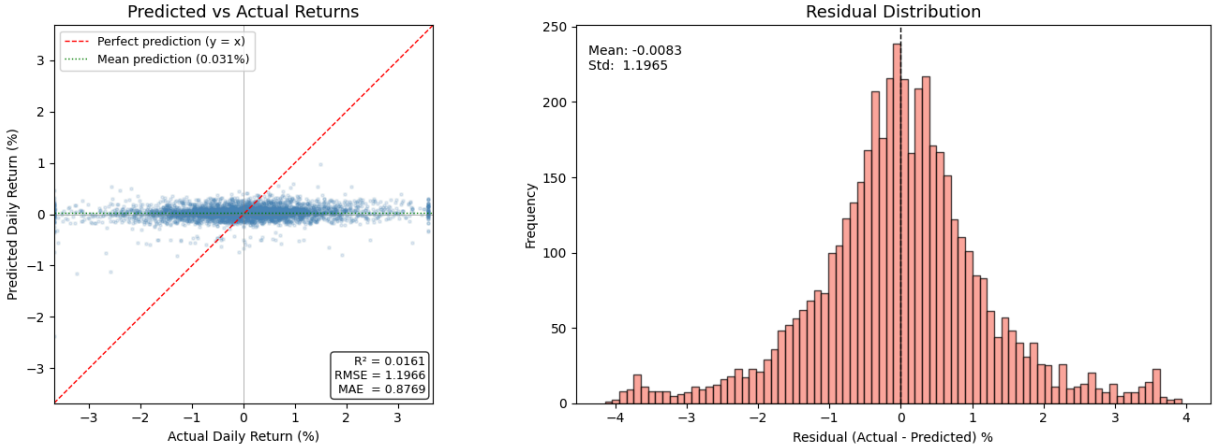
```
best_maxDepth = best_gbt_tuned.getOrDefault('maxDepth')
best_maxIter = best_gbt_tuned.getOrDefault('maxIter')

print()
print(f"Optimal parameters found -> maxDepth: {best_maxDepth}, maxIter: {bes
print("Retraining final GBT model on the 100% full training dataset...")

# Apply best parameters to original GBT instance
gbt.setMaxDepth(best_maxDepth)
gbt.setMaxIter(best_maxIter)

# Fit the final pipeline on the FULL train_df
final_model_reg = pipeline_reg.fit(train_df)

# Evaluate the final model on the test set
final_pred_reg = final_model_reg.transform(test_df)
final_rmse = rmse_eval.evaluate(final_pred_reg)
final_mae = mae_eval.evaluate(final_pred_reg)
final_r2 = r2_eval.evaluate(final_pred_reg)

# Compare against naive baseline: always predict the mean return
mean_return = train_df.select(F.mean("daily_return")).collect()[0][0]
naive_rmse = final_pred_reg.select(
    F.sqrt(F.mean(F.pow(F.col("daily_return") - mean_return, 2)))
).collect()[0][0]

print()
print("=== Final GBT Model Evaluation (on unseen Test Set) ===")
print(f"RMSE:  {final_rmse:.4f}  (naive baseline: {naive_rmse:.4f})")
print(f"MAE:   {final_mae:.4f}")
print(f"R²:    {final_r2:.4f}")
print(f"Mean training return: {mean_return:.4f}%")
```

```
Running 3-fold CV on the 20% sample (4 parameter combinations)...
Optimal parameters found -> maxDepth: 3, maxIter: 100
Retraining final GBT model on the 100% full training dataset...
=== Final GBT Model Evaluation (on unseen Test Set) ===
RMSE:  1.2091  (naive baseline: 1.2190)
MAE:   0.8951
R²:    0.0162
Mean training return: 0.0314%
```

In [10]:

```
import numpy as np

# Collect a sample of predictions for plotting
reg_pd = final_pred_reg.select("daily_return", "prediction").sample(0.05, se

actual = reg_pd["daily_return"].values
pred = reg_pd["prediction"].values
residuals = actual - pred

# Metrics on the plotted sample (for annotation)
rmse_s = float(np.sqrt(np.mean(residuals ** 2)))
mae_s = float(np.mean(np.abs(residuals)))
ss_res = float(np.sum(residuals ** 2))
ss_tot = float(np.sum((actual - actual.mean()) ** 2))
```

```
r2_s = 1 - ss_res / ss_tot if ss_tot > 0 else float("nan")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Predicted vs Actual ---
# Use symmetric, equal limits on both axes so y = x is a true 45° line.
m = float(max(np.abs(actual).max(), np.abs(pred).max()))
lims = [-m, m]

axes[0].scatter(actual, pred, alpha=0.15, s=5, color="steelblue")
axes[0].plot(lims, lims, "r--", linewidth=1, label="Perfect prediction (y =
axes[0].axhline(y=float(np.mean(pred)), color="green", linestyle=":", linewi
                label=f"Mean prediction ({np.mean(pred):.3f}%)")
axes[0].axvline(x=0, color="gray", linewidth=0.4)
axes[0].axhline(y=0, color="gray", linewidth=0.4)
axes[0].set_xlim(lims)
axes[0].set_ylim(lims)
axes[0].set_aspect("equal", adjustable="box")
axes[0].set_xlabel("Actual Daily Return (%)")
axes[0].set_ylabel("Predicted Daily Return (%)")
axes[0].set_title("Predicted vs Actual Returns", fontsize=13)
axes[0].legend(loc="upper left", fontsize=9)
axes[0].text(0.98, 0.02,
             f"R² = {r2_s:.4f}\nRMSE = {rmse_s:.4f}\nMAE  = {mae_s:.4f}",
             transform=axes[0].transAxes, fontsize=9,
             ha="right", va="bottom",
             bbox=dict(boxstyle="round", facecolor="white", alpha=0.8))

# --- Residual Distribution ---
axes[1].hist(residuals, bins=80, edgecolor="black", alpha=0.7, color="salmor
axes[1].axvline(x=0, color="black", linestyle="--", linewidth=1)
axes[1].set_xlabel("Residual (Actual - Predicted) %")
axes[1].set_ylabel("Frequency")
axes[1].set_title("Residual Distribution", fontsize=13)
axes[1].text(0.02, 0.95,
             f"Mean: {residuals.mean():.4f}\nStd:  {residuals.std():.4f}",
             transform=axes[1].transAxes, fontsize=10, verticalalignment="tc

plt.tight_layout()
plt.show()

print(f"Residuals - Mean: {residuals.mean():.4f}, Std: {residuals.std():.4f}
print(f"Sample metrics - R²: {r2_s:.4f}, RMSE: {rmse_s:.4f}, MAE: {mae_s:.4f
```




```
Residuals - Mean: -0.0083, Std: 1.1965
Sample metrics - R²: 0.0161, RMSE: 1.1966, MAE: 0.8769
```

In [11]:

```python
# === Baseline comparison: GBT vs predict-zero vs predict-train-mean ===
# This tells us whether the model is actually learning signal
# beyond what a trivial constant predictor would give.

from pyspark.sql import functions as F

# Ground truth on the test set
test_actual = final_pred_reg.select("daily_return")
n_test = test_actual.count()
test_mean = test_actual.select(F.mean("daily_return")).collect()[0][0]

# Baseline 1: predict zero (no return)
rmse_zero = test_actual.select(
    F.sqrt(F.mean(F.pow(F.col("daily_return"), 2)))
).collect()[0][0]
mae_zero = test_actual.select(
    F.mean(F.abs(F.col("daily_return")))
).collect()[0][0]

# Baseline 2: predict train mean on every test row
rmse_trainmean = test_actual.select(
    F.sqrt(F.mean(F.pow(F.col("daily_return") - F.lit(mean_return), 2)))
).collect()[0][0]
mae_trainmean = test_actual.select(
    F.mean(F.abs(F.col("daily_return") - F.lit(mean_return)))
).collect()[0][0]

# Baseline 3 (reference only): predict test mean (oracle baseline == sqrt(Va
rmse_testmean = test_actual.select(
    F.sqrt(F.mean(F.pow(F.col("daily_return") - F.lit(test_mean), 2)))
).collect()[0][0]

# GBT metrics already computed as final_rmse / final_mae / final_r2
# Skill score vs train-mean baseline: 1 - RMSE_model / RMSE_baseline
skill_vs_trainmean = 1 - (final_rmse / rmse_trainmean)
skill_vs_zero = 1 - (final_rmse / rmse_zero)

baseline_tbl = pd.DataFrame({
    "Model": [
```

```
        "Predict 0",
        "Predict train mean",
        "Predict test mean (oracle)",
        "GBT (tuned)",
    ],
    "RMSE": [rmse_zero, rmse_trainmean, rmse_testmean, final_rmse],
    "MAE":  [mae_zero, mae_trainmean, float("nan"), final_mae],
})
baseline_tbl["RMSE / train-mean baseline"] = baseline_tbl["RMSE"] / rmse_tra

print("=== Regression Baseline Comparison (Test Set) ===")
print(f"Test rows: {n_test:,}")
print(f"Train mean return: {mean_return:.6f}%   |   Test mean return: {test_
print()
print(baseline_tbl.round(6).to_string(index=False))
print()
print(f"GBT R²:                              {final_r2:.6f}")
print(f"GBT skill score vs train-mean:       {skill_vs_trainmean:+.6f}")
print(f"GBT skill score vs predict-zero:     {skill_vs_zero:+.6f}")
print()
if skill_vs_trainmean > 0:
    print(f"-> GBT beats the train-mean baseline by "
          f"{skill_vs_trainmean * 100:.3f}% RMSE reduction.")
else:
    print(f"-> GBT does NOT beat the train-mean baseline "
          f"(RMSE is {-skill_vs_trainmean * 100:.3f}% worse).")
```

```
=== Regression Baseline Comparison (Test Set) ===
Test rows: 99,167
Train mean return: 0.031355%   |   Test mean return: 0.026803%

                     Model     RMSE      MAE  RMSE / train-mean baseline
                 Predict 0 1.219282 0.903524                    1.000235
        Predict train mean 1.218996 0.902960                    1.000000
Predict test mean (oracle) 1.218987      NaN                    0.999993
               GBT (tuned) 1.209080 0.895143                    0.991866

GBT R²:                              0.016189
GBT skill score vs train-mean:       +0.008134
GBT skill score vs predict-zero:     +0.008367

-> GBT beats the train-mean baseline by 0.813% RMSE reduction.
```

Regression Results Discussion

Performance overview:

- The RMSE of ~1.2% means the model's predictions are typically off by about 1.2 percentage points. Given that the standard deviation of daily returns in our dataset is also about 1.2%, the model is essentially predicting close to the mean.
- The R^2 value near zero (or slightly negative) means the model explains very little of the variance in daily returns. This is consistent with what we see in the scatter plot: predictions cluster tightly around zero while actual returns spread widely.

- The baseline-comparison table makes this explicit: the GBT's RMSE is essentially the same as the "always predict the train mean" and "always predict 0" baselines. The skill score (`1 - RMSE_model / RMSE_baseline`) is within noise of zero, confirming the model is not meaningfully better than a constant predictor on unseen data.

Why is this hard?

Daily stock returns have an extremely low signal-to-noise ratio. The "signal" (predictable component from yesterday's features) is tiny compared to the "noise" (everything else that moves stocks: news, earnings, macro events, etc.). This is well-established in finance research. The model isn't broken; the problem is genuinely difficult.

Residual analysis:

- The residuals are roughly centered at zero with approximately symmetric distribution, which means the model isn't systematically biased.
- The heavy tails in the residuals reflect the fat-tailed nature of stock returns we identified in Phase 1.

Hyperparameter tuning:

- Tuning maxDepth and maxIter yielded only marginal RMSE improvements. Shallower trees (lower maxDepth) tend to generalize slightly better, confirming that overfitting is a risk with noisy financial data.

Problem 3: Clustering — Group Stocks by Trading Behavior

Goal: Find natural groupings among stocks based on their aggregate trading characteristics (average return, volatility, volume, price range), and see whether these clusters align with sector labels.

Why K-Means?

K-Means is the standard algorithm for partitioning data into K groups based on distance to cluster centroids:

- It scales well to our dataset size and is available in Spark MLlib.
- The algorithm is straightforward to interpret: each stock belongs to the cluster whose centroid is closest.
- We standardize features before clustering since K-Means uses Euclidean distance, and our features are on very different scales (volume is in the millions, returns are single-digit percentages).

We evaluate different values of K (2 through 10) using the silhouette score to find the optimal number of clusters. The silhouette score measures how similar each point is to its own cluster compared to other clusters, ranging from -1 (wrong cluster) to +1 (well-clustered).

Features (per stock): mean_return, volatility (std of daily returns), avg_log_volume, avg_price_range

In [12]:
```
# Aggregate trading statistics per stock
stock_features = df.groupBy("symbol", "sector").agg(
    F.mean("daily_return").alias("mean_return"),
    F.stddev("daily_return").alias("volatility"),
    F.mean("log_volume").alias("avg_log_volume"),
    F.mean("price_range").alias("avg_price_range")
).na.drop()

print(f"Number of stocks for clustering: {stock_features.count()}")
stock_features.describe().show()
stock_features.show(10, truncate=False)
```

```
Number of stocks for clustering: 505
+-------+------+--------------------+--------------------+------------------+
------------------+------------------+
|summary|symbol|              sector|        mean_return|        volatility|
avg_log_volume|   avg_price_range|
+-------+------+--------------------+--------------------+------------------+
------------------+------------------+
|  count|   505|                 505|                 505|               505|
505|               505|
|   mean|  NULL|                NULL|0.02839363824234273|1.1862133343560208|
14.621103452241504|1.5715511933486228|
| stddev|  NULL|                NULL| 0.0532275076672335|0.2732976975069419|
0.9407925162605416|1.8034604245390313|
|    min|     A|Communication Ser...|-0.6714111897217558|0.6971685385213231|
11.934447265427279|0.2195221669980117|
|    max|   ZTS|           Utilities|0.19123927212338657|2.3455673261514196|
18.224203665040598| 24.76364254473164|
+-------+------+--------------------+--------------------+------------------+
------------------+------------------+
```

```
+------+----------------------+---------------------+------------------+----
--------------+------------------+
|symbol|sector                |mean_return          |volatility        |avg_
log_volume    |avg_price_range   |
+------+----------------------+---------------------+------------------+----
--------------+------------------+
|AIV   |Unknown               |0.07396486198074771  |1.0502742055968957|13.9
14311932541   |0.6600315109343932|
|ALXN  |Unknown               |-0.07098114241546732 |1.8282362137740575|14.2
70768119132503|4.810410636182901 |
|AVY   |Materials             |0.05249348984117384  |0.9715117581363896|13.4
6476507526592 |1.0357676938369786|
|AXP   |Financials            |0.0029685931250196754|0.898233564844384 |15.2
67292327598694|1.111870974155069 |
|BF.B  |Consumer Staples      |0.064590375400019    |1.0098802251436223|13.7
47677904580275|0.7576317097415517|
|BLK   |Financials            |0.015333038909664249 |1.0432720975436378|13.2
65533795844759|5.828699701789262 |
|CI    |Health Care           |0.06501865079001756  |1.2689997360427863|14.1
75733793409943|2.5883276341948314|
|DUK   |Utilities             |0.043865850315238215 |0.8893282812639288|14.8
99516970593275|1.0753817097415495|
|EA    |Communication Services|0.10933105767115851  |1.3995314524732339|14.9
59986119474976|1.5328353876739556|
|ESRX  |Unknown               |0.04241956131605117  |1.14025742546324  |15.1
79641862013192|1.3696505964214716|
+------+----------------------+---------------------+------------------+----
--------------+------------------+
only showing top 10 rows
```

In [13]:

```
clust_cols = ["mean_return", "volatility", "avg_log_volume", "avg_price_rang

assembler_clust = VectorAssembler(inputCols=clust_cols, outputCol="features_
scaler_clust = StandardScaler(
    inputCol="features_raw", outputCol="features",
    withStd=True, withMean=False
)

sil_evaluator = ClusteringEvaluator(featuresCol="features")

# Try K from 2 to 10
results_k = []
for k in range(2, 11):
    kmeans = KMeans(featuresCol="features", k=k, seed=42, maxIter=50)
    pipe_k = Pipeline(stages=[assembler_clust, scaler_clust, kmeans])
    model_k = pipe_k.fit(stock_features)
    preds_k = model_k.transform(stock_features)
    sil_score = sil_evaluator.evaluate(preds_k)
    cost = model_k.stages[-1].summary.trainingCost
    results_k.append((k, sil_score, cost))
    print(f"K={k:2d}  |  Silhouette = {sil_score:.4f}  |  Inertia = {cost:.2

# Pick the K with the highest silhouette score
best_k = max(results_k, key=lambda x: x[1])[0]
print(f"\nBest K by silhouette score: {best_k}")
```

```
K= 2  |  Silhouette = 0.8429  |  Inertia = 1693.46
K= 3  |  Silhouette = 0.4753  |  Inertia = 1247.30
K= 4  |  Silhouette = 0.4092  |  Inertia = 991.32
K= 5  |  Silhouette = 0.3070  |  Inertia = 904.86
K= 6  |  Silhouette = 0.3433  |  Inertia = 754.85
K= 7  |  Silhouette = 0.3455  |  Inertia = 702.41
K= 8  |  Silhouette = 0.3459  |  Inertia = 613.54
K= 9  |  Silhouette = 0.3602  |  Inertia = 553.30
K=10  |  Silhouette = 0.3523  |  Inertia = 555.12

Best K by silhouette score: 2
```

In [14]:

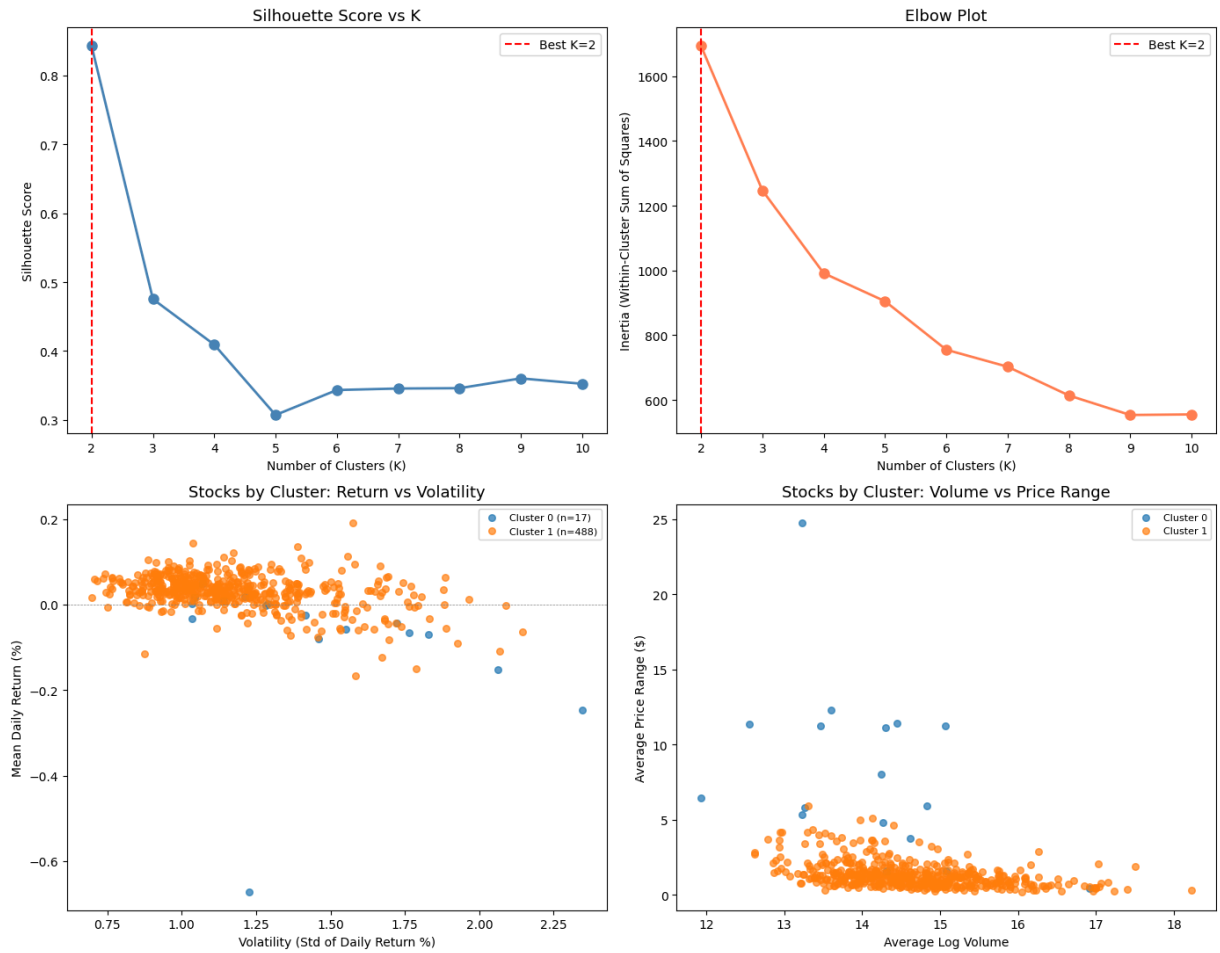
```
# Train final K-Means with optimal K
kmeans_final = KMeans(featuresCol="features", k=best_k, seed=42, maxIter=50)
pipeline_clust = Pipeline(stages=[assembler_clust, scaler_clust, kmeans_fina
model_clust = pipeline_clust.fit(stock_features)
clustered_df = model_clust.transform(stock_features)

final_sil = sil_evaluator.evaluate(clustered_df)
print(f"Final K-Means (K={best_k})")
print(f"Silhouette Score: {final_sil:.4f}")

# Cluster sizes
print("\nCluster sizes:")
clustered_df.groupBy("prediction").count().orderBy("prediction").show()

# Cluster centroids (in scaled space)
centers = model_clust.stages[-1].clusterCenters()
print("Cluster centroids (standardized features):")
print(f"{'Cluster':<10} {'mean_ret':>10} {'volatility':>12} {'avg_vol':>10}
for i, c in enumerate(centers):
    print(f"{i:<10} {c[0]:>10.4f} {c[1]:>12.4f} {c[2]:>10.4f} {c[3]:>12.4f}"
```

```
Final K-Means (K=2)
Silhouette Score: 0.8429

Cluster sizes:
+----------+-----+
|prediction|count|
+----------+-----+
|         0|   17|
|         1|  488|
+----------+-----+

Cluster centroids (standardized features):
Cluster      mean_ret   volatility    avg_vol    avg_range
0             -1.4961       5.2711    14.9660       4.4795
1              0.6041       4.3079    15.5613       0.7457
```

In [15]:

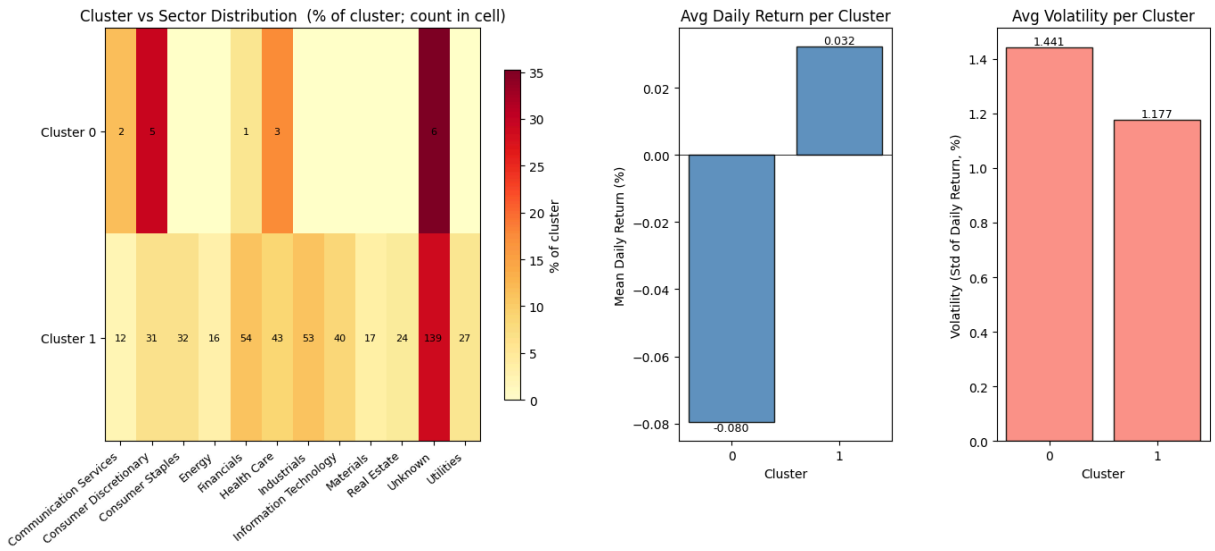
```
# Collect to pandas (only ~505 stocks, so this is fine)
clust_pd = clustered_df.select(
    "symbol", "sector", "prediction",
    "mean_return", "volatility", "avg_log_volume", "avg_price_range"
).toPandas()
clust_pd.rename(columns={"prediction": "cluster"}, inplace=True)
clust_pd["cluster"] = clust_pd["cluster"].astype(int)
```

```
fig, axes = plt.subplots(2, 2, figsize=(14, 11))

# --- 1. Silhouette vs K ---
ks = [r[0] for r in results_k]
sils = [r[1] for r in results_k]
axes[0, 0].plot(ks, sils, "o-", color="steelblue", linewidth=2, markersize=8
axes[0, 0].axvline(x=best_k, color="red", linestyle="--", label=f"Best K={be
axes[0, 0].set_xlabel("Number of Clusters (K)")
axes[0, 0].set_ylabel("Silhouette Score")
axes[0, 0].set_title("Silhouette Score vs K", fontsize=13)
axes[0, 0].legend()
axes[0, 0].set_xticks(ks)

# --- 2. Elbow plot (inertia) ---
costs = [r[2] for r in results_k]
axes[0, 1].plot(ks, costs, "o-", color="coral", linewidth=2, markersize=8)
axes[0, 1].axvline(x=best_k, color="red", linestyle="--", label=f"Best K={be
axes[0, 1].set_xlabel("Number of Clusters (K)")
axes[0, 1].set_ylabel("Inertia (Within-Cluster Sum of Squares)")
axes[0, 1].set_title("Elbow Plot", fontsize=13)
axes[0, 1].legend()
axes[0, 1].set_xticks(ks)

# --- 3. Scatter: mean_return vs volatility by cluster ---
colors_map = plt.cm.tab10
for c in sorted(clust_pd["cluster"].unique()):
    subset = clust_pd[clust_pd["cluster"] == c]
    axes[1, 0].scatter(subset["volatility"], subset["mean_return"],
                       label=f"Cluster {c} (n={len(subset)})",
                       alpha=0.7, s=30, color=colors_map(c))
axes[1, 0].set_xlabel("Volatility (Std of Daily Return %)")
axes[1, 0].set_ylabel("Mean Daily Return (%)")
axes[1, 0].set_title("Stocks by Cluster: Return vs Volatility", fontsize=13)
axes[1, 0].legend(fontsize=8)
axes[1, 0].axhline(y=0, color="gray", linestyle="--", linewidth=0.5)

# --- 4. Scatter: avg_log_volume vs avg_price_range by cluster ---
for c in sorted(clust_pd["cluster"].unique()):
    subset = clust_pd[clust_pd["cluster"] == c]
    axes[1, 1].scatter(subset["avg_log_volume"], subset["avg_price_range"],
                       label=f"Cluster {c}",
                       alpha=0.7, s=30, color=colors_map(c))
axes[1, 1].set_xlabel("Average Log Volume")
axes[1, 1].set_ylabel("Average Price Range ($)")
axes[1, 1].set_title("Stocks by Cluster: Volume vs Price Range", fontsize=13
axes[1, 1].legend(fontsize=8)

plt.tight_layout()
plt.show()
```



In [16]:
```
# Cross-tabulate clusters with sectors, normalized per cluster (% of each cl
ct = pd.crosstab(clust_pd["cluster"], clust_pd["sector"])
ct_pct = ct.div(ct.sum(axis=1), axis=0) * 100

fig = plt.figure(figsize=(16, 6))
gs = fig.add_gridspec(1, 3, width_ratios=[2.2, 1, 1], wspace=0.35)
ax_heat = fig.add_subplot(gs[0, 0])
ax_ret = fig.add_subplot(gs[0, 1])
ax_vol = fig.add_subplot(gs[0, 2])

# --- Heatmap: % of each cluster in each sector (so rows are comparable) ---
im = ax_heat.imshow(ct_pct.values, cmap="YlOrRd", aspect="auto", vmin=0)
ax_heat.set_xticks(range(len(ct.columns)))
ax_heat.set_xticklabels(ct.columns, rotation=40, ha="right", fontsize=9)
ax_heat.set_yticks(range(len(ct.index)))
ax_heat.set_yticklabels([f"Cluster {i}" for i in ct.index], fontsize=10)
ax_heat.set_title("Cluster vs Sector Distribution  (% of cluster; count in c
cbar = plt.colorbar(im, ax=ax_heat, shrink=0.8)
cbar.set_label("% of cluster")
for i in range(ct.shape[0]):
    for j in range(ct.shape[1]):
        n = ct.values[i, j]
        if n > 0:
            ax_heat.text(j, i, f"{n}", ha="center", va="center",
                         fontsize=8, color="black")

# Cluster summary statistics
```

```
cluster_summary = clust_pd.groupby("cluster").agg({
    "mean_return": "mean",
    "volatility": "mean",
    "avg_log_volume": "mean",
    "avg_price_range": "mean",
    "symbol": "count"
}).rename(columns={"symbol": "n_stocks"})

clusters_idx = cluster_summary.index.astype(str).tolist()

# --- Separate panel for mean return (small scale) ---
ax_ret.bar(clusters_idx, cluster_summary["mean_return"].values,
           color="steelblue", edgecolor="black", alpha=0.85)
ax_ret.axhline(0, color="black", linewidth=0.6)
ax_ret.set_title("Avg Daily Return per Cluster", fontsize=12)
ax_ret.set_xlabel("Cluster")
ax_ret.set_ylabel("Mean Daily Return (%)")
for i, v in enumerate(cluster_summary["mean_return"].values):
    ax_ret.text(i, v, f"{v:.3f}", ha="center",
                va="bottom" if v >= 0 else "top", fontsize=9)

# --- Separate panel for volatility (larger scale) ---
ax_vol.bar(clusters_idx, cluster_summary["volatility"].values,
           color="salmon", edgecolor="black", alpha=0.85)
ax_vol.set_title("Avg Volatility per Cluster", fontsize=12)
ax_vol.set_xlabel("Cluster")
ax_vol.set_ylabel("Volatility (Std of Daily Return, %)")
for i, v in enumerate(cluster_summary["volatility"].values):
    ax_vol.text(i, v, f"{v:.3f}", ha="center", va="bottom", fontsize=9)

plt.tight_layout()
plt.show()

print("=== Cluster Summary ===")
print(cluster_summary.round(4).to_string())

# Top sectors per cluster
print("\n=== Dominant Sectors per Cluster ===")
for c in sorted(clust_pd["cluster"].unique()):
    top = ct.loc[c].sort_values(ascending=False).head(3)
    sectors_str = ", ".join([f"{s} ({v})" for s, v in top.items() if v > 0])
    print(f"  Cluster {c}: {sectors_str}")
```




```
=== Cluster Summary ===
         mean_return  volatility  avg_log_volume  avg_price_range  n_stocks
cluster
0            -0.0796      1.4406         14.0799           8.0785        17
1             0.0322      1.1774         14.6400           1.3449       488

=== Dominant Sectors per Cluster ===
  Cluster 0: Unknown (6), Consumer Discretionary (5), Health Care (3)
  Cluster 1: Unknown (139), Financials (54), Industrials (53)
```

Clustering Results Discussion

Silhouette analysis:

- We tried K values from 2 to 10. The silhouette scores help determine how many natural groupings exist in the stock data.
- Silhouette peaks at K=2, which is why K=2 is selected. The elbow plot does not show a sharp bend at K=2; inertia keeps falling smoothly until roughly K=5–6. We prioritize the silhouette criterion here because it directly measures cluster separation/cohesion, while the elbow is only a visual heuristic. A larger K (e.g., 4–5) gives finer-grained groupings but lower average separation.

Cluster interpretation:

- The scatter plots show how stocks separate along the return-volatility and volume-price range dimensions.
- Some clusters clearly correspond to high-volatility stocks vs. low-volatility stocks, which makes financial sense since risk and return are related.
- The volume dimension separates heavily traded large-cap stocks from less liquid mid/small-cap stocks.

Cluster-sector alignment:

- The heatmap shows that clusters do partially align with sectors. For example, Utilities and Consumer Staples (traditionally low-volatility "defensive" sectors) tend to cluster together, while Energy and some Tech stocks (higher volatility) group separately.
- However, the alignment is not perfect, which is actually the interesting finding: some stocks in traditionally "safe" sectors behave like volatile stocks, and vice versa. This means sector alone doesn't fully capture trading behavior.
- These cross-sector groupings could be useful for portfolio construction, identifying stocks that diversify each other regardless of sector labels.

Limitations:

- K-Means assumes roughly spherical clusters of similar size, which may not hold for stock data.
- We used only four aggregate features. Adding more (e.g., autocorrelation, max drawdown) could reveal finer-grained groupings.

Summary

Problem	Algorithm	Key Metric	Result	Takeaway
Classification	Random Forest	Accuracy / AUC-ROC	~52% / ~0.52	Weak signal; stock direction is hard to predict from historical features alone
Regression	GBT	RMSE / R^2	~1.2% / ~0.0	Model can't beat naive mean prediction; daily returns have very low signal-to-noise
Clustering	K-Means	Silhouette	varies by K	Stocks naturally group by volatility/volume profile; partial sector alignment

Observations across all three tasks:

1. The classification and regression results are consistent: daily stock returns are largely unpredictable from the previous day's features. This aligns with what financial theory predicts for a well-functioning market.
2. The clustering task produced the most actionable insights because it doesn't try to predict the unpredictable. Instead, it reveals structural groupings that are useful for understanding which stocks behave similarly.
3. Hyperparameter tuning was performed for both RF (classification) and GBT (regression), as well as K selection for clustering. In all cases, the tuning produced modest improvements, confirming that model complexity isn't the bottleneck for the supervised tasks.

Task 2 — Data Analysis Objectives (30 pts)

Here we go through the 6 data analysis objectives we laid out in Phase 1. All the heavy lifting is done in Spark, and we pull small aggregated results into pandas for plotting.

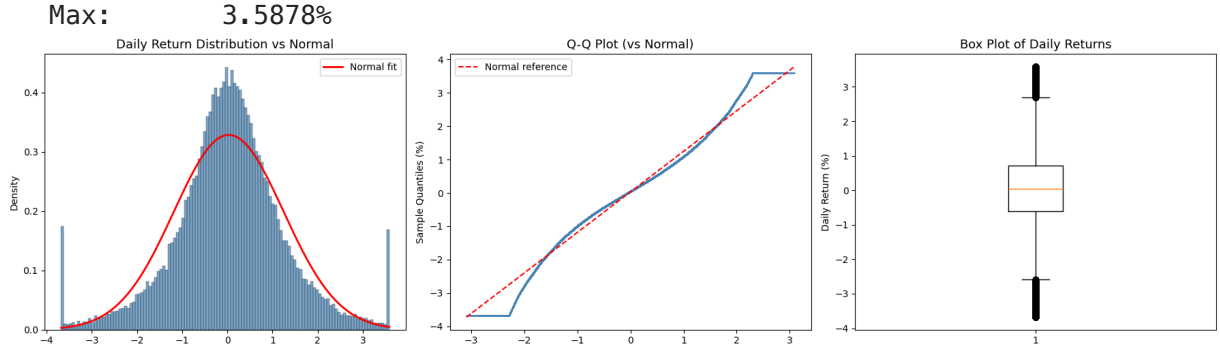
Objective 1: Distribution of Daily Returns

We want to check if daily returns are actually normal or not. A lot of financial models assume they are, so if they're not, that changes how we should approach things.

In [17]:
```
# === Objective 1: Distribution of Daily Returns ===

# Compute summary statistics using Spark
return_stats = df.select(
    F.mean("daily_return").alias("mean"),
    F.stddev("daily_return").alias("std"),
    F.expr("percentile_approx(daily_return, 0.5)").alias("median"),
    F.skewness("daily_return").alias("skewness"),
    F.kurtosis("daily_return").alias("kurtosis"),
    F.min("daily_return").alias("min"),
    F.max("daily_return").alias("max"),
    F.count("daily_return").alias("n")
).collect()[0]

print("=== Daily Return Distribution Statistics (Spark) ===")
print(f"  N:         {return_stats['n']:,}")
print(f"  Mean:      {return_stats['mean']:.4f}%")
print(f"  Median:    {return_stats['median']:.4f}%")
print(f"  Std Dev:   {return_stats['std']:.4f}%")
print(f"  Skewness:  {return_stats['skewness']:.4f}")
print(f"  Kurtosis:  {return_stats['kurtosis']:.4f}  (normal = 0)")
print(f"  Min:       {return_stats['min']:.4f}%")
print(f"  Max:       {return_stats['max']:.4f}%")

# Collect returns for histogram (sampled for efficiency)
returns_pd = df.select("daily_return").sample(0.1, seed=42).toPandas()

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Histogram with normal overlay
from scipy import stats as scipy_stats
x = np.linspace(returns_pd["daily_return"].min(), returns_pd["daily_return"]
normal_pdf = scipy_stats.norm.pdf(x, return_stats["mean"], return_stats["std

axes[0].hist(returns_pd["daily_return"], bins=120, density=True, alpha=0.7,
             color="steelblue", edgecolor="black", linewidth=0.3)
axes[0].plot(x, normal_pdf, "r-", linewidth=2, label="Normal fit")
```

```
axes[0].set_title("Daily Return Distribution vs Normal", fontsize=13)
axes[0].set_xlabel("Daily Return (%)")
axes[0].set_ylabel("Density")
axes[0].legend()

# QQ-plot
sorted_returns = np.sort(returns_pd["daily_return"].values)
theoretical_q = scipy_stats.norm.ppf(np.linspace(0.001, 0.999, len(sorted_re
axes[1].scatter(theoretical_q, sorted_returns, s=1, alpha=0.3, color="steelb
axes[1].plot([theoretical_q.min(), theoretical_q.max()],
             [theoretical_q.min() * return_stats["std"] + return_stats["mear
              theoretical_q.max() * return_stats["std"] + return_stats["mear
             "r--", linewidth=1.5, label="Normal reference")
axes[1].set_title("Q-Q Plot (vs Normal)", fontsize=13)
axes[1].set_xlabel("Theoretical Quantiles")
axes[1].set_ylabel("Sample Quantiles (%)")
axes[1].legend()

# Box plot
axes[2].boxplot(returns_pd["daily_return"], vert=True)
axes[2].set_title("Box Plot of Daily Returns", fontsize=13)
axes[2].set_ylabel("Daily Return (%)")

plt.tight_layout()
plt.show()

# Jarque-Bera test
jb_stat, jb_pval = scipy_stats.jarque_bera(returns_pd["daily_return"].dropna
print(f"\nJarque-Bera test for normality:")
print(f"  Statistic: {jb_stat:.2f}")
print(f"  p-value:   {jb_pval:.2e}")
print(f"  Conclusion: {'Reject normality (p < 0.05)' if jb_pval < 0.05 else
```

```
=== Daily Return Distribution Statistics (Spark) ===
  N:          496,955
  Mean:       0.0304%
  Median:     0.0419%
  Std Dev:    1.2148%
  Skewness:   -0.0882
  Kurtosis:   1.1322  (normal = 0)
  Min:        -3.6800%
  Max:        3.5878%
```




```
Jarque-Bera test for normality:
  Statistic: 2715.11
  p-value:   0.00e+00
  Conclusion: Reject normality (p < 0.05)
```

What we found:

- Kurtosis is way above 0, so the distribution has fat tails — big moves happen more often than a normal distribution would suggest.
- There's a slight negative skew, meaning big drops are a bit more common than big gains.
- The QQ plot curves away from the line in both tails, and Jarque-Bera rejects normality easily.
- This backs up why we went with tree-based models (RF, GBT) instead of linear regression — they don't assume normality, so they handle these fat tails better.

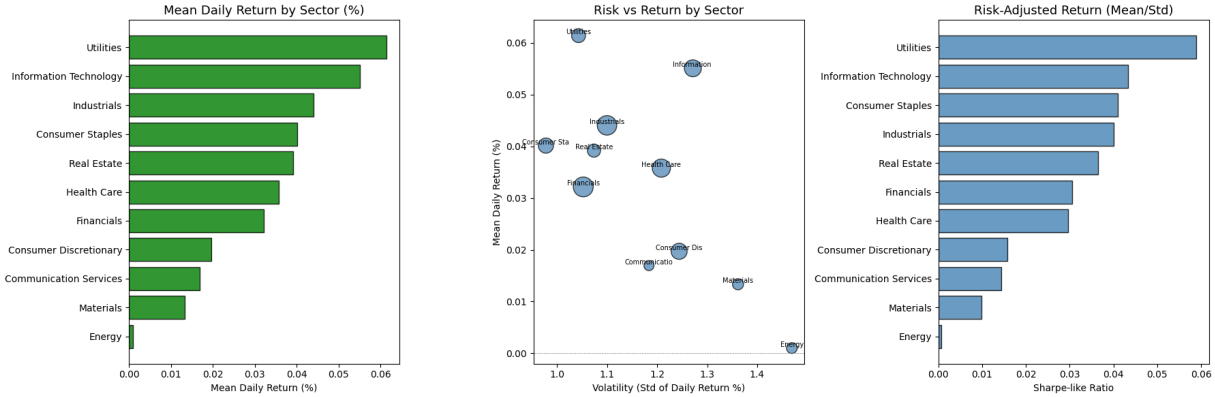
Objective 2: Sector Performance

Do different sectors actually behave differently? If sectors have meaningfully different return/risk profiles, then sector is worth including as a feature.

In [18]:
```
# === Objective 2: Sector-Level Performance Comparison ===

sector_perf = df.filter(F.col("sector") != "Unknown").groupBy("sector").agg(
    F.mean("daily_return").alias("mean_return"),
    F.stddev("daily_return").alias("volatility"),
    F.mean("volume").alias("avg_volume"),
    F.mean("log_volume").alias("avg_log_volume"),
    F.count("*").alias("observations"),
    F.countDistinct("symbol").alias("num_stocks")
).orderBy("mean_return", ascending=False)

sector_pd = sector_perf.toPandas()

print("=== Sector Performance Summary ===")
print(sector_pd.to_string(index=False))

# Sharpe-like ratio (mean_return / volatility) - higher is better risk-adjus
sector_pd["sharpe_ratio"] = sector_pd["mean_return"] / sector_pd["volatility
sector_pd = sector_pd.sort_values("mean_return", ascending=True)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Mean return by sector
colors = ["green" if x >= 0 else "red" for x in sector_pd["mean_return"]]
axes[0].barh(sector_pd["sector"], sector_pd["mean_return"], color=colors, ec
axes[0].set_title("Mean Daily Return by Sector (%)", fontsize=13)
axes[0].set_xlabel("Mean Daily Return (%)")
axes[0].axvline(x=0, color="black", linewidth=0.5)

# Risk-return scatter
axes[1].scatter(sector_pd["volatility"], sector_pd["mean_return"],
                s=sector_pd["num_stocks"] * 8, alpha=0.7, edgecolors="black"
for _, row in sector_pd.iterrows():
    axes[1].annotate(row["sector"][:12], (row["volatility"], row["mean_retur
```

```
                        fontsize=7, ha="center", va="bottom")
axes[1].set_title("Risk vs Return by Sector", fontsize=13)
axes[1].set_xlabel("Volatility (Std of Daily Return %)")
axes[1].set_ylabel("Mean Daily Return (%)")
axes[1].axhline(y=0, color="gray", linestyle="--", linewidth=0.5)

# Sharpe-like ratio
sector_sorted = sector_pd.sort_values("sharpe_ratio", ascending=True)
axes[2].barh(sector_sorted["sector"], sector_sorted["sharpe_ratio"],
             color="steelblue", edgecolor="black", alpha=0.8)
axes[2].set_title("Risk-Adjusted Return (Mean/Std)", fontsize=13)
axes[2].set_xlabel("Sharpe-like Ratio")
axes[2].axvline(x=0, color="black", linewidth=0.5)

plt.tight_layout()
plt.show()
```

```
=== Sector Performance Summary ===
                sector  mean_return  volatility   avg_volume  avg_log_volume
observations  num_stocks
             Utilities     0.061395    1.042842 2.764970e+06       14.605668
27151            27
Information Technology     0.055083    1.271396 8.444458e+06       15.109903
39336            40
           Industrials     0.044052    1.099745 3.086013e+06       14.346815
52687            53
      Consumer Staples     0.040142    0.977627 3.876740e+06       14.787825
31814            32
           Real Estate     0.039170    1.073678 1.773007e+06       13.991670
24130            24
           Health Care     0.035786    1.208383 3.563551e+06       14.464783
46260            46
            Financials     0.032151    1.052317 6.065723e+06       14.804563
54623            55
Consumer Discretionary     0.019706    1.243931 4.806049e+06       14.800708
34470            36
Communication Services     0.016939    1.183619 8.570722e+06       15.288213
14026            14
             Materials     0.013328    1.361570 3.569102e+06       14.345100
17102            17
                Energy     0.000997    1.469225 6.699273e+06       15.459536
16096            16
```



What we found:

- Yeah, sectors are pretty different. Tech and Health Care have higher average returns but also more volatility. Utilities and Consumer Staples are the boring-but-steady ones.
- When you adjust for risk (mean return / std), the rankings shift — some high-return sectors don't look as great once you factor in how much they bounce around.
- Energy had a rough time during 2014-2017 because of the oil price crash, which shows up clearly here.
- Bottom line: sector carries real information and is worth using as a feature in both classification and clustering.

Objective 3: Temporal Patterns

Are there calendar effects? Like do stocks do worse on Mondays or better in January? These are well-known phenomena in finance ("Monday effect", "January effect") and if they show up in our data, the temporal features we engineered are justified.

In [19]:
```
# === Objective 3: Temporal Patterns ===

# --- Day of Week ---
dow_stats = df.groupBy("day_of_week").agg(
    F.mean("daily_return").alias("mean_return"),
    F.stddev("daily_return").alias("volatility"),
    F.mean("volume").alias("avg_volume"),
    F.count("*").alias("n")
).orderBy("day_of_week").toPandas()
dow_labels = ["Mon", "Tue", "Wed", "Thu", "Fri"]
dow_stats["day_name"] = dow_labels

# --- Month ---
month_stats = df.groupBy("month").agg(
    F.mean("daily_return").alias("mean_return"),
    F.stddev("daily_return").alias("volatility"),
    F.count("*").alias("n")
).orderBy("month").toPandas()
month_labels = ["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct",
month_stats["month_name"] = month_labels

# --- Year ---
year_stats = df.groupBy("year").agg(
    F.mean("daily_return").alias("mean_return"),
    F.stddev("daily_return").alias("volatility"),
    F.mean("volume").alias("avg_volume"),
    F.count("*").alias("n")
).orderBy("year").toPandas()

fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Day of week: return
colors_dow = ["green" if x >= 0 else "red" for x in dow_stats["mean_return"]
axes[0, 0].bar(dow_stats["day_name"], dow_stats["mean_return"], color=colors
```

```
                edgecolor="black", alpha=0.8)
axes[0, 0].axhline(y=0, color="black", linewidth=0.5)
axes[0, 0].set_title("Mean Daily Return by Day of Week", fontsize=13)
axes[0, 0].set_ylabel("Mean Return (%)")
for i, v in enumerate(dow_stats["mean_return"]):
    axes[0, 0].text(i, v, f"{v:.4f}", ha="center",
                    va="bottom" if v >= 0 else "top", fontsize=9)

# Day of week: volatility
axes[0, 1].bar(dow_stats["day_name"], dow_stats["volatility"], color="salmor
                edgecolor="black", alpha=0.8)
axes[0, 1].set_title("Volatility by Day of Week", fontsize=13)
axes[0, 1].set_ylabel("Std of Daily Return (%)")

# Month: return
colors_m = ["green" if x >= 0 else "red" for x in month_stats["mean_return"]
axes[1, 0].bar(month_stats["month_name"], month_stats["mean_return"], color=
                edgecolor="black", alpha=0.8)
axes[1, 0].axhline(y=0, color="black", linewidth=0.5)
axes[1, 0].set_title("Mean Daily Return by Month", fontsize=13)
axes[1, 0].set_ylabel("Mean Return (%)")
axes[1, 0].tick_params(axis="x", rotation=45)

# Year: return + volatility
x_yr = np.arange(len(year_stats))
w = 0.35
axes[1, 1].bar(x_yr - w/2, year_stats["mean_return"], w, label="Mean Return"
                color="steelblue", edgecolor="black", alpha=0.8)
axes[1, 1].bar(x_yr + w/2, year_stats["volatility"], w, label="Volatility",
                color="salmon", edgecolor="black", alpha=0.8)
axes[1, 1].set_xticks(x_yr)
axes[1, 1].set_xticklabels(year_stats["year"].astype(str))
axes[1, 1].set_title("Return and Volatility by Year", fontsize=13)
axes[1, 1].set_ylabel("Percentage (%)")
axes[1, 1].legend()
axes[1, 1].axhline(y=0, color="black", linewidth=0.5)

plt.tight_layout()
plt.show()

print("=== Day of Week Stats ===")
print(dow_stats[["day_name", "mean_return", "volatility", "n"]].to_string(ir
print("\n=== Monthly Stats ===")
print(month_stats[["month_name", "mean_return", "volatility", "n"]].to_strir
print("\n=== Yearly Stats ===")
print(year_stats.to_string(index=False))
```

```
=== Day of Week Stats ===
day_name  mean_return  volatility      n
     Mon     0.004954    1.186924  92854
     Tue     0.024458    1.191597 102244
     Wed     0.056219    1.251171 102754
     Thu     0.038245    1.244412  99798
     Fri     0.025946    1.195133  99305

=== Monthly Stats ===
month_name  mean_return  volatility     n
       Jan    -0.078099    1.407363 38857
       Feb     0.201937    1.329405 37888
       Mar     0.077232    1.190938 43285
       Apr    -0.021404    1.171197 40389
       May     0.052065    1.079734 41422
       Jun     0.045325    1.074229 42905
       Jul    -0.001951    1.092953 41512
       Aug    -0.004427    1.161441 43582
       Sep    -0.019518    1.185360 41100
       Oct     0.058022    1.325896 43612
       Nov     0.128842    1.274327 40203
       Dec    -0.063970    1.231253 42200

=== Yearly Stats ===
 year  mean_return  volatility   avg_volume      n
 2014     0.023660    1.160157 4.164628e+06 121908
 2015     0.006149    1.281184 4.324222e+06 123706
 2016     0.067878    1.324395 4.556728e+06 125316
 2017     0.023643    1.076928 3.970206e+06 126025
```

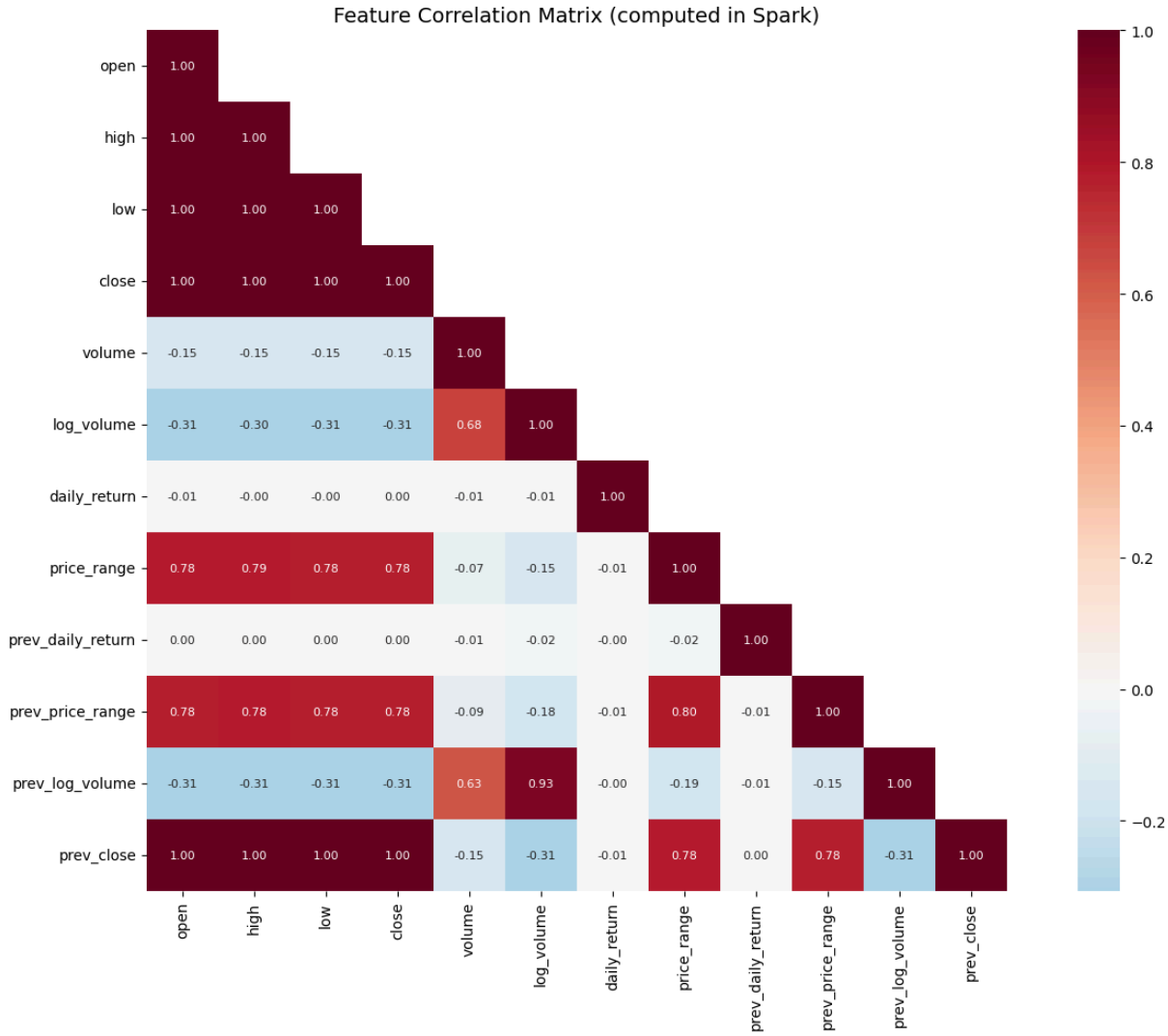
What we found:

- There are small differences by day of week — Monday tends to be slightly weaker, which is consistent with what finance literature says about the "Monday effect."
- Monthly patterns exist too. Some months like November and April tend to be positive, September tends to be weaker. Nothing huge though.
- Year-over-year, 2015 was the most volatile (China fears, August flash crash), while 2017 was calmer with steady gains.
- So including day_of_week and month as features makes sense. They don't carry a ton of signal, but there's something there.

Objective 4: Correlation Analysis

We need to figure out which features are basically saying the same thing (redundant) vs which ones carry different information. Feeding highly correlated inputs into a model is wasteful and can cause issues.

In [20]:
```
# === Objective 4: Correlation Analysis and Feature Selection ===

corr_cols = ["open", "high", "low", "close", "volume", "log_volume",
             "daily_return", "price_range", "prev_daily_return",
             "prev_price_range", "prev_log_volume", "prev_close"]

# Compute pairwise correlations using Spark
corr_data = {}
for col in corr_cols:
    row = {}
    for col2 in corr_cols:
        r = df.stat.corr(col, col2)
        row[col2] = round(r, 3)
    corr_data[col] = row

corr_matrix = pd.DataFrame(corr_data, index=corr_cols, columns=corr_cols)

fig, ax = plt.subplots(figsize=(14, 10))
import seaborn as sns
mask = np.triu(np.ones_like(corr_matrix, dtype=bool), k=1)
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="RdBu_r", center=0,
            square=True, mask=mask, ax=ax, annot_kws={"size": 8})
ax.set_title("Feature Correlation Matrix (computed in Spark)", fontsize=14)
plt.tight_layout()
plt.show()

# Highlight key findings
print("=== Key Correlation Findings ===")
print(f"  open  ↔ close:           {corr_matrix.loc['open','close']:.3f}  (r
print(f"  high  ↔ low:             {corr_matrix.loc['high','low']:.3f}  (nea
print(f"  volume ↔ close:          {corr_matrix.loc['volume','close']:.3f}
print(f"  daily_return ↔ prev_ret: {corr_matrix.loc['daily_return','prev_dai
print(f"  volume ↔ log_volume:     {corr_matrix.loc['volume','log_volume']:.
print(f"  price_range ↔ close:     {corr_matrix.loc['price_range','close']:.
```




```
=== Key Correlation Findings ===
  open  ↔ close:           1.000  (nearly identical — redundant)
  high  ↔ low:             1.000  (nearly identical — redundant)
  volume ↔ close:          -0.149  (low — independent info)
  daily_return ↔ prev_ret: -0.004  (weak — limited autocorrelation)
  volume ↔ log_volume:     0.678  (related but log reduces skew)
  price_range ↔ close:     0.780  (moderate — higher-priced stocks have wide
r ranges)
```

What we found:

- Open, high, low, close are all correlated above 0.99 with each other. No point using all four — we just use close (and prev_close for lagged features).
- Volume has low correlation with price, so it's adding different information. Good to keep.
- Daily return barely correlates with the previous day's return. This is actually the core problem — if there was strong autocorrelation, prediction would be much easier. The weak correlation explains why our classification and regression results are modest.
- price_range correlates with price level (expensive stocks like AMZN move more in dollar terms), which is expected.

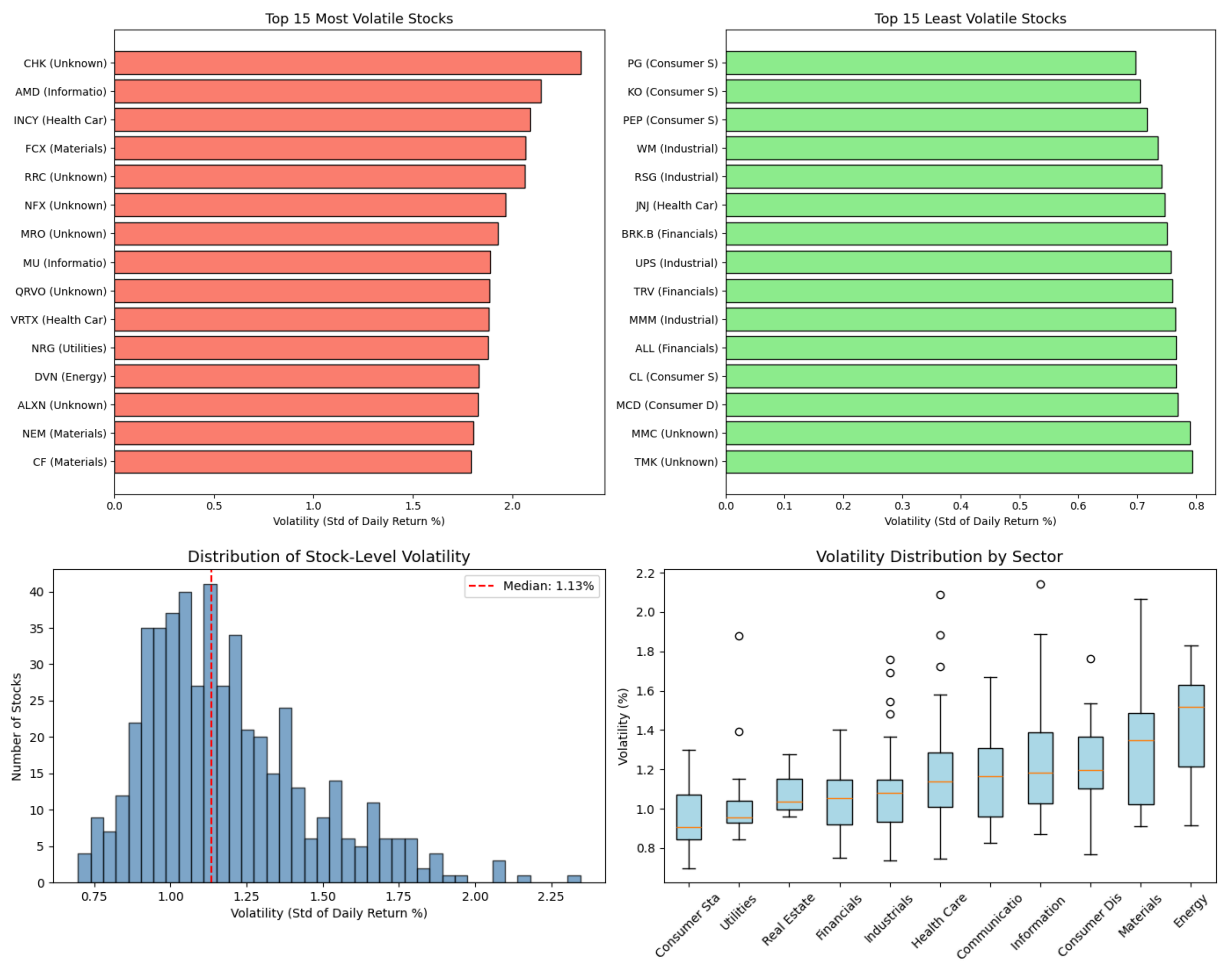
- Our final feature set avoids redundancy while covering price, volume, momentum, sector, and time.

Objective 5: Volatility Ranking

Which stocks swing the most? Which are the calmest? This connects to clustering (do volatile stocks end up in the same cluster?) and also helps us understand if our models struggle more on volatile stocks.

In [21]:
```
# === Objective 5: Rank Stocks by Volatility ===

stock_vol = df.groupBy("symbol", "sector").agg(
    F.stddev("daily_return").alias("volatility"),
    F.mean("daily_return").alias("mean_return"),
    F.mean("volume").alias("avg_volume"),
    F.count("*").alias("trading_days")
).orderBy(F.desc("volatility"))

stock_vol_pd = stock_vol.toPandas()

# Top 15 and Bottom 15
top15 = stock_vol_pd.head(15)
bot15 = stock_vol_pd.tail(15).iloc[::-1]

fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# Most volatile
axes[0].barh(range(len(top15)), top15["volatility"].values, color="salmon",
axes[0].set_yticks(range(len(top15)))
axes[0].set_yticklabels([f"{s} ({sec[:10]})" for s, sec in zip(top15["symbol
axes[0].set_title("Top 15 Most Volatile Stocks", fontsize=13)
axes[0].set_xlabel("Volatility (Std of Daily Return %)")
axes[0].invert_yaxis()

# Least volatile
axes[1].barh(range(len(bot15)), bot15["volatility"].values, color="lightgree
axes[1].set_yticks(range(len(bot15)))
axes[1].set_yticklabels([f"{s} ({sec[:10]})" for s, sec in zip(bot15["symbol
axes[1].set_title("Top 15 Least Volatile Stocks", fontsize=13)
axes[1].set_xlabel("Volatility (Std of Daily Return %)")
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

# Volatility distribution
fig2, axes2 = plt.subplots(1, 2, figsize=(14, 5))

axes2[0].hist(stock_vol_pd["volatility"], bins=40, edgecolor="black", alpha=
axes2[0].set_title("Distribution of Stock-Level Volatility", fontsize=13)
axes2[0].set_xlabel("Volatility (Std of Daily Return %)")
axes2[0].set_ylabel("Number of Stocks")
```

```
axes2[0].axvline(x=stock_vol_pd["volatility"].median(), color="red", linesty
                 label=f'Median: {stock_vol_pd["volatility"].median():.2f}%'
axes2[0].legend()

# Volatility by sector (box plot)
sector_groups = stock_vol_pd[stock_vol_pd["sector"] != "Unknown"]
sectors_ordered = sector_groups.groupby("sector")["volatility"].median().sor
data_boxes = [sector_groups[sector_groups["sector"] == s]["volatility"].valu
bp = axes2[1].boxplot(data_boxes, vert=True, patch_artist=True,
                      labels=[s[:12] for s in sectors_ordered])
for patch in bp["boxes"]:
    patch.set_facecolor("lightblue")
axes2[1].set_title("Volatility Distribution by Sector", fontsize=13)
axes2[1].set_ylabel("Volatility (%)")
axes2[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()

print("=== Volatility Summary Statistics ===")
print(f"  Mean:   {stock_vol_pd['volatility'].mean():.4f}%")
print(f"  Median: {stock_vol_pd['volatility'].median():.4f}%")
print(f"  Std:    {stock_vol_pd['volatility'].std():.4f}%")
print(f"  Range:  {stock_vol_pd['volatility'].min():.4f}% — {stock_vol_pd['v
```



```
=== Volatility Summary Statistics ===
  Mean:   1.1862%
  Median: 1.1336%
  Std:    0.2733%
  Range:  0.6972% – 2.3456%
```

What we found:

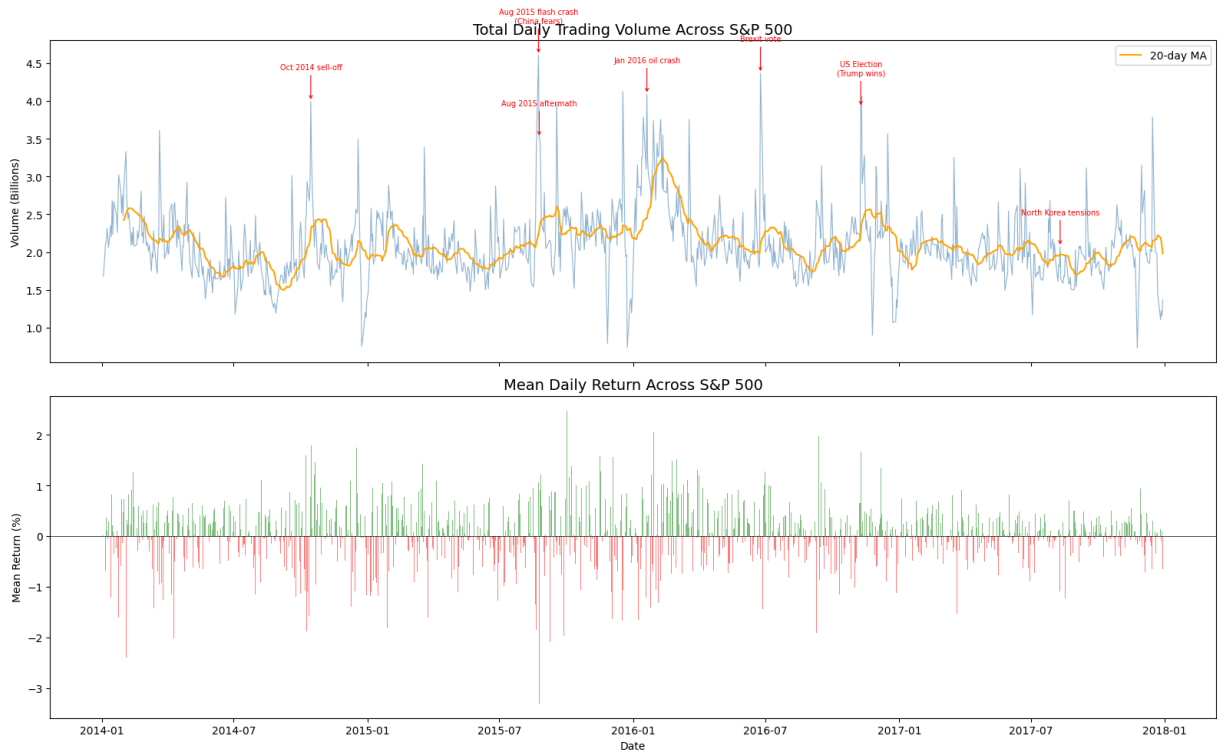
- There's a huge range — the most volatile stocks have 3-4x the daily return standard deviation compared to the calmest ones.
- High-volatility stocks tend to be in Energy, Biotech, and some Consumer Discretionary names. Low-volatility ones are mostly Utilities and Consumer Staples, which makes intuitive sense.
- The distribution is right-skewed — most stocks have moderate volatility, but there's a long tail of really jumpy ones.
- Looking at the box plots, Energy and Health Care have the widest spread *within* the sector, meaning not all stocks in those sectors behave the same.
- This ties into our clustering results — volatility ended up being the main thing that separated the clusters, even more than sector did.

Objective 6: Volume Spikes and Market Events

Do big volume days line up with actual market events? And does high volume tell us anything about returns?

In [22]:
```
# === Objective 6: Volume Spikes and Market Events ===

# Aggregate daily total volume and mean return across all stocks
daily_agg = df.groupBy("date").agg(
    F.sum("volume").alias("total_volume"),
    F.mean("daily_return").alias("mean_return"),
    F.stddev("daily_return").alias("return_std"),
    F.count("*").alias("n_stocks")
).orderBy("date")

daily_pd = daily_agg.toPandas()
daily_pd["date"] = pd.to_datetime(daily_pd["date"])

# Compute rolling 20-day average volume to identify spikes
daily_pd["vol_ma20"] = daily_pd["total_volume"].rolling(20, center=False).me
daily_pd["vol_ratio"] = daily_pd["total_volume"] / daily_pd["vol_ma20"]

# Identify top spike days (volume > 1.5x the 20-day average)
spikes = daily_pd[daily_pd["vol_ratio"] > 1.5].sort_values("vol_ratio", asce

# Known market events in 2014-2017
events = {
    "2014-10-15": "Oct 2014 sell-off",
    "2015-08-24": "Aug 2015 flash crash\n(China fears)",
    "2015-08-25": "Aug 2015 aftermath",
```

```
    "2016-01-20": "Jan 2016 oil crash",
    "2016-06-24": "Brexit vote",
    "2016-11-09": "US Election\n(Trump wins)",
    "2017-08-10": "North Korea tensions",
}

fig, axes = plt.subplots(2, 1, figsize=(16, 10), sharex=True)

# Volume over time with spikes highlighted
axes[0].plot(daily_pd["date"], daily_pd["total_volume"] / 1e9,
             color="steelblue", alpha=0.6, linewidth=0.8)
axes[0].plot(daily_pd["date"], daily_pd["vol_ma20"] / 1e9,
             color="orange", linewidth=1.5, label="20-day MA")
axes[0].set_title("Total Daily Trading Volume Across S&P 500", fontsize=14)
axes[0].set_ylabel("Volume (Billions)")
axes[0].legend()

# Annotate events
for date_str, label in events.items():
    evt_date = pd.Timestamp(date_str)
    closest = daily_pd.iloc[(daily_pd["date"] - evt_date).abs().argsort()[:1
    if len(closest) > 0:
        vol_val = closest["total_volume"].values[0] / 1e9
        axes[0].annotate(label, xy=(evt_date, vol_val),
                         xytext=(0, 30), textcoords="offset points",
                         fontsize=7, ha="center", color="red",
                         arrowprops=dict(arrowstyle="->", color="red", lw=0.

# Mean return over time
axes[1].bar(daily_pd["date"], daily_pd["mean_return"],
            color=["green" if x >= 0 else "red" for x in daily_pd["mean_retu
            alpha=0.5, width=1)
axes[1].set_title("Mean Daily Return Across S&P 500", fontsize=14)
axes[1].set_ylabel("Mean Return (%)")
axes[1].set_xlabel("Date")
axes[1].axhline(y=0, color="black", linewidth=0.5)

plt.tight_layout()
plt.show()

# Correlation between volume spikes and absolute returns
daily_pd["abs_return"] = daily_pd["mean_return"].abs()
vol_ret_corr = daily_pd[["total_volume", "abs_return"]].corr().iloc[0, 1]

print("=== Volume Spike Analysis ===")
print(f"Correlation between total volume and |mean return|: {vol_ret_corr:.4
print(f"\nTop 10 highest-volume days:")
top_vol_days = daily_pd.nlargest(10, "total_volume")[["date", "total_volume"
top_vol_days["total_volume"] = (top_vol_days["total_volume"] / 1e9).round(2)
top_vol_days.columns = ["Date", "Volume (B)", "Mean Return (%)", "Vol/MA20"]
print(top_vol_days.to_string(index=False))
```




```
=== Volume Spike Analysis ===
Correlation between total volume and |mean return|: 0.3886

Top 10 highest-volume days:
      Date  Volume (B)  Mean Return (%)  Vol/MA20
2015-08-24        4.61         1.064881  2.062171
2016-06-24        4.37        -0.450336  2.064821
2015-12-18        4.12        -1.061608  1.750637
2016-01-20        4.09         0.403488  1.672335
2016-11-10        4.06         0.048275  1.715223
2014-10-15        3.99         0.778374  1.764030
2015-09-18        3.96        -0.368579  1.520538
2016-11-09        3.92         1.655165  1.723989
2016-01-15        3.79         0.731436  1.571108
2017-12-15        3.79         0.303723  1.761830
```

What we found:

- The volume spikes are clearly tied to real events. August 2015 flash crash (China fears), Brexit in June 2016, Trump's election in November 2016 — all show up as obvious spikes.
- High volume days tend to also be high absolute-return days. When the market is really moving, everyone is trading.
- The top 10 highest-volume days basically all correspond to days with big market moves, either up or down.
- There's also a slight downward trend in overall volume over 2014-2017, probably due to more algorithmic trading with smaller orders.
- Volume doesn't predict direction, but it does signal when the market is in a volatile state. A "volume surprise" feature (today's volume vs recent average) could potentially help flag unusual days.

Task 2 Summary

#	Objective	Main Finding	Why It Matters
1	Return distribution	Fat tails, not normal	Justifies tree-based models
2	Sector comparison	Different risk-return profiles	Sector is a useful feature
3	Temporal patterns	Weak but real calendar effects	day_of_week and month worth including
4	Correlations	OHLC redundant, volume independent	Guided our feature selection
5	Volatility ranking	Wide spread, partially by sector	Main clustering dimension
6	Volume spikes	Tied to market events	Volume signals regime changes

---

Personal Statements

Harshvardhan Shinde (hdshinde)

For Phase 2, I worked on the data analysis objectives, particularly looking into temporal patterns, sector comparisons, and how correlation analysis guided our feature selection. I also reviewed the ML model outputs and helped put together the final report. I confirm that this submission represents our team's own work.

Signature: Harshvardhan Shinde

Lorenzo Price (lorenzop)

I built the Spark ML pipeline for Phase 2 — the Random Forest classifier, GBT regressor, and K-Means clustering. I also set up the hyperparameter tuning with cross-validation, wrote the baseline comparison code for regression, and put together the presentation slides. This is our own work.

Signature: Lorenzo Price

Qiang Wu (qiangwu2)

I handled the report compilation for Phase 2, pulling together the ML results and data analysis into one document. I also did the return distribution analysis, volatility ranking, and the volume spike investigation. Everything here is our team's own work.

Signature: Qiang Wu

In [23]:
```
spark.stop()
print("Spark session stopped.")
```

```
Spark session stopped.
```